Prompt-Aware WiFi Scheduling for Generative AI Services

Shugang Hao
Singapore University of Technology and Design (SUTD)
Singapore, Singapore
shugang_hao@sutd.edu.sg

Lingjie Duan*
Hong Kong University of Science and Technology
Guangzhou, China
lingjieduan@hkust-gz.edu.cn

Abstract

Mobile devices are increasingly accessing generative AI (GenAI) services through edge-deployed Large Language Models (LLMs) over WiFi networks, particularly in smart homes and smart offices. Devices transmit uplink prompts via OFDMA, a key WiFi 6 technology that enables concurrent access and improves network efficiency for dense GenAI traffic. The edge server batches these prompts to maximize inference efficiency. However, existing WiFi scheduling algorithms (e.g., max-weight and round-robin) largely ignore non-negligible LLM inference delay, random and diverse among different prompt inputs. Further, the batch inference delay is dictated by the longest prompt, coupling the service latency of co-batched stations and destabilizing device-side queues. In this work, we propose a novel prompt-aware WiFi scheduling for GenAI services. We formulate the scheduling problem as a Markov Decision Process (MDP) and prove its NP-hardness. We design a Renewal Drift-Plus-Penalty (RDPP) policy to balance backlogs against batching costs, using the drift to ensure queue stability and the penalty to constrain frame durations. We prove that RDPP is throughput-optimal and achieves a near-optimal average queueing delay. To further address the impact of output response length on LLM inference, we extend to develop a robust R-RDPP policy that accounts for output-length uncertainty, still achieving a small optimality gap. We conduct experiments on an Nvidia A100 GPU (80GB) to measure LLM inference latency and derive closed-form functions of batch size and token lengths. NS-3 experiments further demonstrate great improvements of our proposed policies over existing WiFi schedulers in service delay, queue length, and tail latency.

CCS Concepts

• **Networks** → **Wireless local area networks**; • **Mathematics of computing** → **Mathematical optimization**.

Keywords

WiFi for Generative AI, OFDMA scheduling, LLM inference

ACM Reference Format:
Shugang Hao and Lingjie Duan. 2018. Prompt-Aware WiFi Scheduling for Generative AI Services. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX).* ACM, New York, NY, USA, 20 pages. https://doi.org/XXXXXXX.XXXXXXX

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Mobile devices are increasingly accessing generative AI (GenAI) services through edge-deployed Large Language Models (LLMs) over WiFi networks, where indoor environments (e.g., smart homes and smart offices) serve as the primary deployment domains (e.g., [30], [14]). In a typical edge GenAI workflow, a prompt is first transmitted from the station to a WiFi access point (AP) over the air interface via orthogonal frequency division multiple access (OFDMA), a key technology in WiFi 6 and 7 that significantly improves network efficiency, especially in dense environments for GenAI (e.g., [17], [20]). The AP then forwards the prompt over a wired backhaul to a nearby co-located edge LLM server, which generates the response to the request (e.g., [28], [16]).

Unlike traditional wireless traffic (e.g., web browsing or video streaming) where the delay bottleneck lies in the air-interface transmission (e.g., [25], [26]), this new GenAI traffic incurs non-negligible server-side inference delay compared to wireless transmission delay (e.g., [24], [30]), random and diverse among different prompt inputs. Further, the batch inference delay is dictated by the longest prompt (e.g., [14], [10]), coupling the service latency of co-batched stations and destabilizing device-side queues. Existing WiFi scheduling algorithms (e.g., max-weight style and round-robin) largely ignore these inference characteristics and make decisions based solely on queue backlogs and channel conditions (e.g., [27], [11], [23], [19]). One research question naturally arises:

- *Q1. How do current WiFi scheduling algorithms perform when facing generative AI services?*

Later we prove that existing methods lead to unstable station queues, as a single long prompt bottlenecks the batch-inference process. This necessitates a new scheduling policy accounting for both air-interface transmissions and server-side inference costs. Few studies in the recent wireless literature analyze the end-to-end latency of GenAI services. Kim *et al.* [14] optimize decisions between local processing and server offloading by balancing cost, latency, and response quality for LLM services over 5G networks. Yang *et al.* [30] characterize end-to-end latency distributions using queueing models that capture both air-interface and LLM inference delays. Jin *et al.* [13] propose an end-to-end coordination framework CORA to improve the fraction of latency-critical inference requests meeting their latency targets. However, they primarily employ content-agnostic or bit-rate-centric models and fail to account for the non-linear coupling inherent in LLM batching, where a batch's service time is dictated by the maximum prompt length. Thus, the interplay between OFDMA sub-carrier allocation and batch-inference efficiency remains unexplored.

Existing LLM works study inference delay and batching behavior from the server-side perspective. Yang *et al.* [31] apply queueing-theoretic abstractions to analyze how output length distribution affects inference delay. Guldogan *et al.* [10] propose binning-based batching to group requests by output length and improve throughput. Yu *et al.* [32] introduce Orca to improve GPU utilization under dynamic workloads. More recently, prompt-aware server-side schedulers have emerged that predict output lengths to approximate shortest-job-first ordering (e.g., [8]), further improving inference latency. However, all these works optimize inference latency strictly from a server-side perspective, ignoring source-side queue dynamics and providing no queue stability guarantees at the station side. Moreover, they do not account for wireless transmission delays, treating request arrivals as exogenous rather than as outcomes of wireless scheduling decisions. In practice, the AP's scheduling and the server's batch inference are tightly coupled, yet the impact of this interaction on station-side queue stability remains unexplored.

It is challenging to design efficient WiFi scheduling for GenAI services. First, prompt arrivals from mobile devices are stochastic in both input length and arrival time, making it difficult to balance the immediate relief of queue backlogs against the heavy inference delays. Second, the edge LLM batching mechanism induces a strong coupling effect, where the inference delay is dictated by the maximum input and output lengths within a batch. Thus, the scheduling of one station's prompt directly affects the service of others. This max-based cost fundamentally differs from classical air-interface scheduling, where server delay is typically negligible and user performance remains independent, making it difficult to adapt traditional models to handle this non-linear coupling. The second research question is thus:

- *Q2. How to design an efficient prompt-aware WiFi scheduling policy for GenAI services to stabilize queue backlogs?*

We summarize our key novelty and main results as follows.

- *Prompt-aware WiFi scheduling for generative AI services:* In this paper, we study a novel prompt-aware WiFi scheduling for generative AI services. Unlike existing WiFi scheduling approaches that consider only air-interface transmission and ignore server-side inference delay, we explicitly incorporate LLM inference characteristics induced by prompt batching. We aim to provide new analytical insights into *how to design efficient scheduling schemes for generative AI services.*
- *MDP formulation and NP-hardness:* We prove that existing WiFi scheduling algorithms (e.g., max-weight and round-robin) can lead to unstable queues at the station side under GenAI workloads. We formulate prompt-aware OFDMA scheduling as a Markov Decision Process (MDP). We prove that computing an optimal policy for this MDP is NP-hard.
- *RDPP policy with theoretical guarantees:* We propose a Renewal Drift-Plus-Penalty (RDPP) scheduling policy that jointly optimizes queue backlogs and batching-induced LLM inference costs, where the drift acts to stabilize the queues by clearing accumulated backlogs and the penalty restrains excessive frame durations. We prove that RDPP is throughput-optimal and achieves a near-optimal long-run average waiting delay with gap $O(1/V)$ for a tunable parameter $V > 0$.

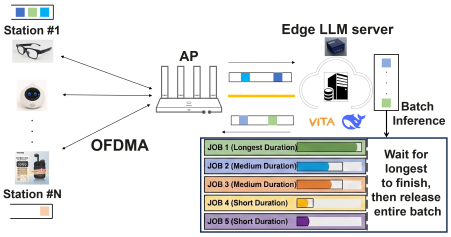

Figure 1: System model of WiFi for generative AI services.

- *Extension to output-length uncertainty:* We further extend RDPP to account for uncertainty in output response length, which affects decoding latency and is unknown at scheduling time. We propose a robust R-RDPP policy and prove that it still achieves a small optimality gap. We conduct experiments on an Nvidia A100 GPU (80GB) to measure LLM inference latency, and derive closed-form functions of batch size and token lengths. NS-3 experiments further demonstrate great improvements of our proposed policies over existing methods in queue length, service delay, and tail latency.

The rest of this paper is organized as follows. Section 2 introduces the system model, problem formulation, and benchmark policies' instability under GenAI workloads. Section 3 presents the MDP formulation and proves its NP-hardness. Section 4 details our RDPP policy design and provides theoretical guarantees on stability and near-optimality. Section 5 extends RDPP to handle output length uncertainty. Section 6 conducts experiments to validate our theoretical findings. Section 7 finally concludes the paper. Due to the page limit, we skip some detailed proofs but still introduce sketches in the main text. Please refer to online technical report [12] for details.

2 System Model and Problem Formulation

In this section, we first introduce our system model of WiFi for GenAI services. We then give our problem formulation and the instability of existing WiFi scheduling algorithms.

2.1 System Model of WiFi for GenAI

As shown in Figure 1, we consider a time-slotted WiFi-based edge AI system, where an access point (AP) schedules and serves uplink prompt transmissions from N stations (e.g., smart-home devices like smartphones and smart security cameras, or office devices such as laptops and smart displays) over the air interface. Meanwhile, it forwards received prompts' packets via a wired backhaul to a nearby co-located edge LLM server for inference and response generation. In each time slot t, each station $i \in [N] := \{1, \ldots, N\}$ generates a new prompt with input length $\ell_i^{\text{in}} \in \{L_0^{\text{in}}, L_1^{\text{in}}, \ldots, L_I^{\text{in}}\}$ (in terms of packet number), where $L_0^{\text{in}} = 0$ means no prompt generated and $0 < L_1^{\text{in}} < \cdots < L_I^{\text{in}}$ with $Pr(\ell_i^{\text{in}} = L_j^{\text{in}}) = \alpha_j \geq 0$ and $\sum_{j=0}^{I} \alpha_j = 1$ i.i.d. across stations and time slots. This i.i.d. assumption reflects uncoordinated user behavior, where diverse devices generate heterogeneous tasks stochastically based on immediate user needs. Thus, the total expected packet arrival rate across the system is given by $\Lambda = \sum_{i=1}^{N} \mathbb{E}[\ell_i^{\text{in}}] = N \sum_{j=1}^{I} \alpha_j L_j^{\text{in}}$. In Section 5, we extend to consider a continuous interval of prompt input lengths.

Each station maintains a buffer to store prompts awaiting transmission and inference, allowing flexible service ordering as determined by the scheduling policy. Buffering occurs only at the station side. The AP and the edge LLM server do not maintain persistent buffers and process prompts immediately upon reception. By making uplink scheduling aware of the server's real-time workload, the system effectively maintains a virtually centralized queue at the station side, avoiding multi-hop queueing delays.

The AP employs OFDMA to schedule uplink transmissions. In each time slot, the AP can allocate up to $k \geq 1$ resource units (RUs) to serve up to k stations in parallel. When station i is scheduled, one prompt from its buffer is transmitted to the AP and forwarded to the edge LLM server. A length-ℓ^{in} prompt requires ℓ^{in} slots to complete transmission. We consider time-homogeneous channel conditions across stations to focus on the fundamental interaction between prompt-aware scheduling and batch inference delay. This is consistent with the standard approach in the wireless scheduling theory literature for GenAI services (e.g., [29], [4], [33]).

The edge LLM server processes incoming prompts using static batching, widely adopted in the LLM inference literature (e.g., [32], [31]). In each service round, the server groups multiple prompts into a batch and performs joint inference, where the batch size is constrained by hardware resources. We consider $B = k$ to reflect a common deployment scenario where the edge server's batching capacity is provisioned to match the OFDMA parallelism of the WiFi AP. This alignment ensures that all prompts delivered in a single wireless frame can be processed in a single inference round, avoiding unnecessary queueing at the server-side buffer.

A key property of LLM batch inference is that the inference delay depends on the longest prompt input and the longest response output in the batch (e.g., [31], [32]). Let $\ell^{\text{in}}_{\max} := \max_{i \in \mathcal{B}} \ell^{\text{in}}_i$ and $\ell^{\text{out}}_{\max} := \max_{i \in \mathcal{B}} \ell^{\text{out}}_i$, where ℓ^{out}_i denotes the output response length generated by the LLM for station i. We model the Time-To-First-Token (TTFT) as follows:

$$\text{TTFT} = f(B, \ell^{\text{in}}_{\max}), \tag{1}$$

where $f(\cdot)$ is known and nondecreasing in both arguments, capturing the empirical observation that prefill latency scales with the maximum input length in the batch (e.g., [24], [7]). The decode-phase latency, characterized by the inter-token latency or Time-Per-Output-Token (TPOT), depends on $\ell^{\text{out}}_{\max}$ and is modeled by a known and nondecreasing function $g(\cdot)$ as follows:

$$\text{TPOT} = g(B, \ell^{\text{in}}_{\max}, \ell^{\text{out}}_{\max}). \tag{2}$$

We define the frame duration of a batch as

$$\tau := \ell^{\text{in}}_{\max} + f(B, \ell^{\text{in}}_{\max}) + g(B, \ell^{\text{in}}_{\max}, \ell^{\text{out}}_{\max}) \times (\ell^{\text{out}}_{\max} - 1), \tag{3}$$

capturing the total time from the start of OFDMA transmission to the completion of response generation. Note that the latency functions $f(\cdot)$ and $g(\cdot)$ are intrinsic to the LLM inference hardware and remain invariant to wireless channel conditions, as they depend solely on the batch size and token lengths processed by the GPU. In practice, both functions can be estimated beforehand via offline profiling on the edge LLM server. In Section 6.1, we conduct experiments on an Nvidia A100 GPU to measure TTFT and TPOT, and derive closed-form functions of batch size and token lengths.

Table 1: Summary of Key Notations

Symbol	Description
N, k, B	Num. of STAs, OFDMA RUs, batch size ($B{=}k$)
$t, \Delta t$	Time slot index; slot duration
$\ell^{\text{in}}_i(t), \ell^{\text{out}}_i$	Input prompt / output response length of STA i
$L^{\text{in}}_0, \ldots, L^{\text{in}}_I$	Discrete prompt length levels ($L^{\text{in}}_0 = 0$: no arrival)
α_j, Λ	Prompt length probability; total arrival rate
$Q_i(t), Q^+_i(t)$	Backlog / post-arrival backlog at STA i
$\boldsymbol{Q}(t)$	Vector of all station backlogs
$\mathcal{B}(t), n_{i,j}(t)$	Buffer composition; num. prompts of length L^{in}_j
$r(t)$	Residual service time of ongoing batch
$\ell^{\text{in}}_{\max}, \ell^{\text{out}}_{\max}$	Max input / output length in a batch
$f(\cdot), g(\cdot)$	TTFT (prefill latency); TPOT (decode latency)
$\tau(L)$	Frame duration
$\mathcal{S}^{\text{sched}}_t, U_i(t)$	Scheduled station set; scheduling indicator
$H^{\text{sel}}_i(t), m(t)$	Selected prompt length; num. scheduled STAs
$\ell^{\text{sel}}_{\max}(t)$	Max selected prompt length in batch
S_t	MDP state: $(\boldsymbol{Q}(t), \mathcal{B}(t), r(t))$
$c^\star, \bar{Q}^\pi$	Optimal average cost; avg. backlog under π
$\mu^\star, \mathcal{C}$	Max service rate; capacity region
$L, \mathcal{L}$	Batch threshold; feasible threshold set
$\ell_i(L), S(L)$	Longest prompt $\leq L$ at STA i; selected batch
$V, \Phi_V(L)$	Tunable parameter; RDPP score
$\delta, \mathcal{G}_\delta$	Quantization resolution; grid
$Q_\delta(\cdot), I_\delta$	Quantization map; num. grid points
$\hat{\tau}(L), \hat{\sigma}_\tau(L)$	Estimated frame duration; its std. dev.
η, Δ_g	Robustness param.; decode duration spread
$\Phi^\eta_V(L), \mu^\star_\delta$	R-RDPP score; quantized capacity

2.2 Problem Formulation and Criteria

Following the WiFi scheduling literature (e.g., [27], [9], [6]), we aim to minimize queueing backlog at the station or device side by appropriately designing wireless scheduling decisions. Let $Q_i(t)$ denote the packet number in station i's buffer at the beginning of slot t. A scheduling policy π determines, at each slot, which subset of stations is scheduled for OFDMA transmission. Our objective is to minimize the long-run average total buffer length:

$$\min_{\pi} \quad \bar{Q}^\pi := \limsup_{T \to \infty} \frac{1}{T} \mathbb{E}_\pi \left[\sum_{t=0}^{T-1} \sum_{i=1}^{N} Q_i(t) \right]. \tag{4}$$

By Little's law, minimizing $\bar{Q}^\pi$ is equivalent to minimizing the average queueing delay, making this formulation appropriate for interactive generative AI services.

Before formalizing the stability analysis, we first define the notion of capacity region and then characterize the stability.

Definition 2.1 (Capacity Region). The capacity region $\mathcal{C}$ is defined as the set of total packet arrival rates Λ for which there exists a scheduling policy that stabilizes the system. Equivalently, $\Lambda \in \mathcal{C}$ if and only if $\Lambda < \mu^\star$, where $\mu^\star$ denotes the maximum achievable service rate over all feasible batch compositions.

Definition 2.2 (Stability). The system is *stable* under policy π if it leads to a finite expected queue length $\bar{Q}^\pi < \infty$. A policy that achieves stability for all arrival rates strictly inside the capacity region is called *throughput-optimal* (e.g., [27], [5]).

2.3 Preliminary: Benchmark Schemes

In this subsection, we introduce the commonly used max-weight style and round-robin scheduling as benchmark schemes to examine their performance under generative AI workloads.

At each scheduling opportunity, max-weight style scheduling (e.g., [27]) assigns a weight to each station proportional to its queue backlog and selects the set of stations that maximizes the total weight subject to resource constraints. In an OFDMA system with k parallel resource units, this reduces to selecting the k stations with the largest queue lengths. More recent variants incorporate multi-agent reinforcement learning (MFMAPPO [11]) and deep hierarchical reinforcement learning (DHRL [23]) to learn scheduling weights for 802.11ax OFDMA, yet they still rely on queue backlogs and channel conditions without accounting for batch inference coupling. Round-robin scheduling is widely used for its simplicity and fairness (e.g., WiFi 6, 7 [2], [18]). The policy cycles through stations in a fixed order and allocates transmission opportunities evenly across users, independent of queue backlogs or prompt characteristics. Both fail to ensure stability under GenAI workloads.

Lemma 2.3. *Under max-weight style or round-robin scheduling, the system is unstable for all $\Lambda < \mu^\star$ in the capacity region, where*

$$\mu^\star := \max_{\ell \in \{L_1^{\text{in}}, \ldots, L_I^{\text{in}}\}} \frac{k\ell}{\ell + f(k,\ell)} \tag{5}$$

denotes the maximum achievable service rate over all feasible batch compositions, and $k\ell/(\ell + f(k,\ell))$ represents the maximum aggregate payload normalized by the joint transmission-inference duration.

The root cause of instability under both max-weight and round-robin policies is the inefficient mixing of long and short prompts, which triggers a max-based inference delay. While max-weight prioritizes long prompts due to larger backlogs and round-robin is simply length-oblivious, both frequently include a long prompt in every batch. Since inference time is governed by the longest request, this persistent mixing inflates service times and reduces the effective service rate, eventually causing queues to grow unboundedly.

Since existing WiFi schedulers are highly inefficient under GenAI workloads, this motivates prompt-aware OFDMA scheduling that jointly accounts for wireless transmission and LLM inference to ensure queue stability. In Sections 3–4, we focus on the prefill latency by setting $g \equiv 0$ (e.g., real-time interactive chatbots and streaming voice assistants). We incorporate the analysis of TPOT with output-length uncertainty (e.g., code generation) in Section 5.

3 MDP Formulation and NP-Hardness

In this section, we formulate the prompt-aware OFDMA scheduling problem for WiFi-based GenAI services as a Markov Decision Process (MDP), which captures stochastic prompt arrivals, batch-dependent LLM inference delay, and sequential scheduling decisions. We then characterize the optimality conditions of this MDP and prove that computing an optimal policy is NP-hard.

3.1 MDP Formulation

The system evolves in discrete time slots where prompt arrivals occur at the beginning. In the following, we illustrate our design of state, action, transition and cost in detail.

State. In time slot t, the system state is defined as

$$S_t := \big(\boldsymbol{Q}(t), \boldsymbol{\mathcal{B}}(t), r(t)\big).$$

We denote $\boldsymbol{Q}(t) = (Q_1(t), \ldots, Q_N(t))$ as buffer backlogs at all stations and $Q_i(t) \in \mathbb{Z}_{\geq 0}$ represents the total number of buffered prompts' packets at station i. We represent the buffer composition state at time t by $\boldsymbol{\mathcal{B}}(t) := \{n_{i,j}(t) : i \in [N],\ j \in [I]\}$, where $n_{i,j}(t) \in \mathbb{Z}_{\geq 0}$ denotes the number of buffered prompts at station i with input length L_j^{in}. $r(t) \in \mathbb{Z}_{\geq 0}$ is the residual service time of the ongoing batch, capturing the remaining service time before the edge server releases the responses. The server is idle only if $r(t) = 0$ and a new scheduling decision can be made. Recall that $\ell_i^{\text{in}}(t)$ denotes the input length (in packets) of the prompt arriving at station i at the beginning of slot t. The post-arrival backlog is

$$Q_i^+(t) = Q_i(t) + \ell_i^{\text{in}}(t).$$

Action. At a decision epoch (i.e., when $r(t) = 0$ and at least one station has $Q_i^+(t) > 0$), the scheduler selects a subset

$$S_t^{\text{sched}} \subseteq \{1, \ldots, N\}, \qquad |S_t^{\text{sched}}| \leq k,$$

and chooses one prompt's packets to transmit from each selected station. If $r(t) > 0$ or all stations are empty after arrivals, the only admissible action is to remain idle.

Transition. Define the scheduling indicator $U_i(t) = \mathbf{1}\{i \in S_t^{\text{sched}}\}$ with $\sum_{i=1}^N U_i(t) \leq k$. Denote $H_i^{\text{sel}}(t)$ as the input length of the selected prompt. The buffer dynamics evolves as

$$Q_i(t+1) = Q_i^+(t) - U_i(t)\, H_i^{\text{sel}}(t).$$

If a new batch is initiated at slot t, define

$$m(t) := \sum_{i=1}^N U_i(t), \qquad \ell_{\max}^{\text{sel}}(t) := \max_{i: U_i(t)=1} H_i^{\text{sel}}(t).$$

The corresponding frame duration is

$$\tau(t) = \ell_{\max}^{\text{sel}}(t) + f\big(m(t), \ell_{\max}^{\text{sel}}(t)\big).$$

The residual service time updates as

$$r(t+1) = \begin{cases} \tau(t) - 1, & \text{if } r(t) = 0 \text{ and } m(t) \geq 1, \\ \max\{r(t) - 1, 0\}, & \text{otherwise.} \end{cases}$$

Objective. The per-slot cost is defined as the total backlog,

$$c(S_t, a_t) = \sum_{i=1}^N Q_i(t).$$

For a policy π, the long-run average backlog is

$$c^\pi = \limsup_{T \to \infty} \frac{1}{T} \mathbb{E}_\pi \left[\sum_{t=0}^{T-1} c(S_t, a_t) \right]. \tag{6}$$

Note that the c^π in (6) is consistent with the objective $\bar{Q}^\pi$ defined in (4). Our objective is to find a policy π minimizing c^π. The backlog objective in (6) is jointly shaped by both wireless transmission and LLM batch inference. Since new scheduling decisions are made only when the server is idle, the batch inference delay directly governs how frequently station queues can be drained, causing arriving prompts to accumulate during server-busy periods.

3.2 NP-Hardness

We first characterize the optimality conditions of the above MDP. Under standard regularity assumptions for countable-state average-cost MDPs (e.g., [3]), there exists an optimal average cost $c^\star$ and

a differential cost function $h : \mathcal{S} \to \mathbb{R}$ satisfying the average-cost optimality equation (ACOE):

$$h(S) + c^\star = \min_{a \in \mathcal{A}(S)} \{c(S, a) + \mathbb{E}[h(S') \mid S, a]\}, \tag{7}$$

where $\mathcal{A}(S)$ denotes the admissible action set in state S, and S' is the next state under the transition dynamics. Although the ACOE characterizes optimal policies, solving it exactly is computationally intractable as shown below.

Lemma 3.1. *Computing an optimal scheduling policy for the MDP in Section 3.1 is NP-hard, i.e., unless* P = NP*, there exists no polynomial-time algorithm to find an optimal solution for all problem instances.*

Proof Sketch. We reduce from the NP-hard single-machine batch scheduling problem $1 \mid p\text{-batch}, s_j = 1 \mid \sum C_j$ (e.g., [1]). Here, a single server (1) groups uniform-capacity jobs (p-batch, $s_j = 1$) to minimize total completion time ($\sum C_j$), where a batch's processing time equals its longest job. Our formulation directly generalizes this max-based combinatorial structure by incorporating coupled wireless transmission delays.

Given this NP-hardness result, it is computationally intractable to obtain an exact optimal policy for the formulated MDP in general. This motivates us to develop a low-complexity and near-optimal scheduling policy with provable performance guarantees, which will be presented in the next section.

4 RDPP: A Near-Optimal Scheduling Policy

In this section, we develop a low-complexity scheduling policy with provable performance guarantees. We first introduce the design principles of the Renewal Drift-Plus-Penalty (RDPP) policy by exploiting the renewal structure induced by batch processing. We then establish its theoretical guarantees on throughput-optimal stability and near-optimal average backlog performance.

4.1 RDPP Policy Design

As shown in Section 3.2, computing an optimal scheduling policy is NP-hard. We therefore seek a low-complexity policy that stabilizes the system whenever possible and achieves near-optimal average queueing backlog. Our design leverages the renewal structure induced by batch processing. Scheduling decisions are made only at renewal epochs (i.e., when $r(t) = 0$), while the system evolves deterministically during each busy frame.

At a renewal epoch t, the scheduler observes the post-arrival state $\{Q_i^+(t), n_{i,j}^+(t)\}$. To reduce combinatorial complexity, we parameterize each candidate batch by its maximum input-length threshold L. Fixing L determines the frame duration τ in (3) as

$$\tau(L) := L + f(k, L),$$

and restricts eligible prompts to those with length not exceeding L. For each station i, define

$$\ell_i(L) := \max\{\ell \le L : \text{a prompt of length } \ell \text{ exists at station } i\},$$

with $\ell_i(L) = 0$ if no such prompt exists. For each feasible threshold L, we construct a batch $S(L)$ by selecting up to k stations with the largest weights, denoted as follows:

$$\big(Q_i^+(t) + V(\tau(L) - 1)\big)\ell_i(L),$$

Algorithm 1 RDPP Scheduling Policy

Require: Stations N, resource units k, prompt-length thresholds $\mathcal{L} = \{L_1^{\text{in}}, \ldots, L_I^{\text{in}}\}$, tradeoff parameter $V > 0$
Ensure: Scheduling decisions $\{L^\star, S(L^\star)\}$ at each renewal epoch
for each renewal epoch (server idle $r(t) = 0$, some buffer nonempty $Q_i^+(t) > 0$) **do**
  for each candidate max-length threshold $L \in \mathcal{L}$ **do**
    Frame duration: $\tau(L) \leftarrow L + f(k, L)$
    Longest eligible prompt at each STA: $\ell_i(L) \leftarrow \max\{\ell \le L :$ prompt of length ℓ at STA $i\}$
    Per-station weight (backlog relief × prompt size): $w_i(L) \leftarrow (Q_i^+(t) + V(\tau(L) - 1)) \cdot \ell_i(L), \forall i$
    Select batch with highest weights: $S(L) \leftarrow$ top-k STAs by $w_i(L)$
    Evaluate per-slot reward: compute $\Phi_V(L)$ via (8)
  Pick best threshold: $L^\star \leftarrow \arg\max_{L \in \mathcal{L}} \Phi_V(L)$
  Transmit batch $S(L^\star)$ via OFDMA; update queues
  Begin batch inference: $r(t+1) \leftarrow \tau(L^\star) - 1$; all responses released together at $t + \tau(L^\star)$

where $V > 0$ is a tunable parameter. The RDPP score is then

$$\Phi_V(L) := \frac{\sum_{i \in S(L)} \big(Q_i^+(t) + V(\tau(L) - 1)\big)\ell_i(L)}{\tau(L)} - \frac{V\Lambda}{2}(\tau(L) - 1). \tag{8}$$

The structure of $\Phi_V(L)$ reflects a renewal drift-plus-penalty principle. The numerator rewards backlog reduction weighted by prompt length, while division by $\tau(L)$ converts the reward into a per-slot rate. The final term penalizes long frames that allow excessive arrival accumulation. The parameter V controls the tradeoff between backlog responsiveness and frame efficiency.

Given the renewal structure of the system and the threshold-based parameterization of feasible batches, we now formally present the complete RDPP scheduling procedure in Algorithm 1.

The per-renewal computational complexity of RDPP is $O(I\,N \log k)$, since at most I candidate thresholds are evaluated and, for each threshold, selecting up to k stations with largest weights can be implemented using a size-k heap over N stations.

4.2 Theoretical Guarantees

Having specified the RDPP policy, we now analyze its stability properties. We first show RDPP achieves throughput-optimal stability, i.e., it stabilizes the system whenever stabilization is feasible.

Lemma 4.1 (Throughput-Optimal Stability). *For any $V > 0$, RDPP stabilizes the system for all arrival rates strictly inside the whole capacity region $\Lambda < \mu^\star$.*

Lemma 4.1 shows that RDPP preserves the full capacity region of the system. Next, we quantify the delay performance of RDPP relative to the optimal policy.

Theorem 4.2 (Near-Optimality). *Let $c^\star$ denote the optimal long-run average cost of the MDP in Section 3.1. The long-run average backlog $c^{\text{RDPP}(V)}$ under our RDPP policy in Algorithm 1 satisfies*

$$c^{\text{RDPP}(V)} \le c^\star + O\left(\frac{1}{V}\right).$$

Further, the time-average total backlog remains bounded as follows:

$$\limsup_{T \to \infty} \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E}\left[\sum_{i=1}^{N} Q_i(t)\right] = O(1).$$

Theorem 4.2 captures the fundamental performance tradeoff of the RDPP policy. As in (8), the parameter V acts as a control "knob" that dictates how the scheduler balances two competing forces, i.e., clearing the current queue quickly (backlog responsiveness) versus selecting batch thresholds that optimize the server's time (frame efficiency). By explicitly designing the RDPP rule to maximize this renewal drift-plus-penalty score, scaling the efficiency terms by V guarantees that the deviation from the theoretical minimum cost $c^\star$ is tightly bounded by $O(1/V)$. Therefore, while V must remain finite to maintain queue awareness, increasing V allows the optimality gap of RDPP to be made arbitrarily small.

Unlike classical drift-plus-penalty formulations in network utility maximization where a fundamental $[O(1/V), O(V)]$ tradeoff arises, our objective directly minimizes backlog. Thus, increasing V improves the optimality gap without inflating queue lengths, and the backlog scaling remains $O(1)$. This structural distinction arises because the penalty term in our RDPP score (8) penalizes long frames rather than trading off a separate utility objective against delay, allowing both goals to align as V grows.

5 Extension to Output Response Uncertainty

In this section, we first extend the system model to account for output-response uncertainty related to TPOT in (2), and extend to continuously distributed input prompt lengths beyond the discrete set in Section 2.1. We then design a robust quantized scheduling policy and establish its theoretical performance guarantees.

5.1 Extension of System Model to Output Response Uncertainty

With output lengths taken into account, the frame duration τ of a batch $\mathcal{B}$ in (3) becomes

$$\tau(B, \ell^{\text{in}}_{\max}, \ell^{\text{out}}_{\max}) = \ell^{\text{in}}_{\max} + f(B, \ell^{\text{in}}_{\max}) + g(B, \ell^{\text{in}}_{\max}, \ell^{\text{out}}_{\max}),$$

where $\ell^{\text{in}}_{\max} = \max_{i\in\mathcal{B}} \ell^{\text{in}}_i$ and $\ell^{\text{out}}_{\max} = \max_{i\in\mathcal{B}} \ell^{\text{out}}_i$. The first two terms correspond to the transmission plus prefill duration and are known at the renewal epoch, whereas the decode term depends on output lengths that are revealed only after batch selection. This creates an information asymmetry that the scheduler must commit to a batch while part of its service time is still random.

We assume each prompt's output length ℓ^{out} is drawn from a bounded-support distribution that may depend on its input length, and output lengths are conditionally independent across prompts given input lengths. The variability induced by output uncertainty is captured by

$$\Delta_g := \max_{b\le k,\ \ell^{\text{in}}} \left[\max_{\ell^{\text{out}}} g(b, \ell^{\text{in}}, \ell^{\text{out}}) - \min_{\ell^{\text{out}}} g(b, \ell^{\text{in}}, \ell^{\text{out}}) \right],$$

which measures the worst-case spread of the decode duration due to unknown output response lengths. Intuitively, a larger Δ_g implies that two batches with identical observed input profiles can still incur very different decode times, making purely input-based scheduling less reliable. We further generalize input prompt lengths from a finite set to a bounded continuum $\ell^{\text{in}} \in [\ell^{\text{in}}_{\min}, \ell^{\text{in}}_{\max}]$, drawn from a general distribution. Bounded support ensures uniformly bounded frame durations and preserves the renewal drift structure. However, continuous lengths make the state description more complex. More importantly, they induce an infinite action space because the batch threshold can take any real value. This motivates quantization to recover a finite decision space with controlled approximation error.

Algorithm 2 R-RDPP: Robust Extension with Output Uncertainty

Require: Stations N, resource units k, quantization resolution $\delta > 0$, robustness parameter $\eta \ge 0$, tradeoff parameter $V > 0$
Ensure: Scheduling decisions $\{L^\star, S(L^\star)\}$ at each renewal epoch
Discretize prompt lengths into grid $\mathcal{G}_\delta$; collect runtime estimates of mean decode latency $\mathbb{E}[g(\cdot)]$ and its variability $\hat{\sigma}_\tau(\cdot)$
for each renewal epoch (server idle $r(t) = 0$, some buffer nonempty $Q^+_i(t) > 0$) **do**
  Map each prompt to nearest grid point: $\ell^{\text{in}}_i \mapsto Q_\delta(\ell^{\text{in}}_i), \forall i$
  for each candidate threshold $L \in \mathcal{G}_\delta$ **do**
    Estimated frame duration (with expected decode time): $\hat{\tau}(L) \leftarrow L + f(|S(L)|, L) + \mathbb{E}[g(\cdot) \mid S(L)]$
    Per-station weight: $w_i(L) \leftarrow (Q^+_i(t) + V(\hat{\tau}(L) - 1)) \cdot \ell^{\text{in}}_i, \forall i$
    Select batch with highest weights: $S(L) \leftarrow$ top-k STAs by $w_i(L)$
    Evaluate uncertainty-aware reward: compute $\Phi^\eta_V(L)$ via (9)
  Pick best threshold: $L^\star \leftarrow \arg\max_{L\in\mathcal{G}_\delta} \Phi^\eta_V(L)$
  Transmit batch $S(L^\star)$ via OFDMA; update queues with *true* (unquantized) ℓ^{in}_i
  Begin batch inference: $r(t+1) \leftarrow \tau_{\text{actual}}(L^\star) - 1$; all responses released together

5.2 Robust RDPP and Performance Guarantees

To handle both continuous input lengths and output-induced service-time uncertainty, we design a robust quantized extension of RDPP. The key idea is to approximate the continuous prompt-length space using a finite grid while incorporating uncertainty-aware scoring to hedge against unknown decode durations.

We fix a resolution $\delta > 0$ and define the uniform grid

$$\mathcal{G}_\delta := \left\{ \ell^{\text{in}}_{\min} + m\delta : m = 0, 1, \ldots, \left\lfloor \frac{\ell^{\text{in}}_{\max} - \ell^{\text{in}}_{\min}}{\delta} \right\rfloor \right\}.$$

For any prompt with true input length $\ell^{\text{in}} \in [\ell^{\text{in}}_{\min}, \ell^{\text{in}}_{\max}]$, define the quantization map

$$Q_\delta(\ell^{\text{in}}) := \max\{L \in \mathcal{G}_\delta : L \le \ell^{\text{in}}\}.$$

Then $Q_\delta(\ell^{\text{in}}) \le \ell^{\text{in}} < Q_\delta(\ell^{\text{in}}) + \delta$, and each prompt is assigned to one of $I_\delta = |\mathcal{G}_\delta| = O(1/\delta)$ discrete types. Quantization is used only for scheduling decisions (e.g., threshold selection and batch construction), while the actual queue evolution and served packet counts are computed using the true input lengths.

Since the decode duration is unknown at scheduling time, we replace the deterministic frame length in RDPP by an estimated duration. For a candidate threshold $L \in \mathcal{G}_\delta$ and its constructed batch $S(L)$, define

$$\hat{\tau}(L) = L + f(|S(L)|, L) + \mathbb{E}\left[g(|S(L)|, L, \ell^{\text{out}}_{\max}) \mid S(L)\right].$$

To hedge against output uncertainty, we inflate the effective duration and define the score

$$\Phi^\eta_V(L) = \frac{\sum_{i\in S(L)} \left(Q^+_i(t) + V(\hat{\tau}(L) - 1)\right)\ell^{\text{in}}_i}{\hat{\tau}(L) + \eta\hat{\sigma}_\tau(L)} - \frac{V\Lambda}{2}(\hat{\tau}(L) - 1), \quad (9)$$

where $\hat{\sigma}_\tau(L)$ denotes the standard deviation of the random frame duration under batch $S(L)$ and satisfies $\hat{\sigma}_\tau(L) \le \Delta_g$, and $\eta \ge 0$ is a robustness parameter. This robustness term discourages batches whose service times exhibit large uncertainty, thereby preventing

systematic selection of high-variance batches that appear efficient only in expectation. Note that the policy only requires empirical estimates of the mean and variability of the decode latency, which can be obtained from runtime measurements, and does not require the full distribution of output token lengths.

Based on the quantized representation and the robust score defined above, we present the complete R-RDPP scheduling procedure in Algorithm 2. Compared to RDPP (Algorithm 1), the key modifications are: (i) input lengths are quantized onto the grid $\mathcal{G}_\delta$; (ii) the frame duration is estimated using the expected decode time $\hat{\tau}(L)$; and (iii) the score $\Phi_V^\eta(L)$ incorporates the robustness term $\eta\hat{\sigma}_\tau(L)$ to hedge against output-length uncertainty.

The per-decision computational complexity is $O(I_\delta N \log k)$ with $I_\delta = O(1/\delta)$, since at most I_δ thresholds are evaluated and, for each threshold, selecting up to k stations with the largest weights can be implemented using a size-k heap over N stations. This matches the $O(I\,N \log k)$ complexity of RDPP, with I replaced by the finer grid size I_δ, offering a controllable resolution–complexity tradeoff.

We now establish the theoretical guarantees of R-RDPP. The following result characterizes its stability region.

Theorem 5.1 (Asymptotically Throughput-Optimal Stability). *For any $V > 0$, $\delta > 0$, and $\eta \geq 0$, R-RDPP stabilizes the system for all arrival rates satisfying*

$$\Lambda < \mu_\delta^\star, \qquad \mu_\delta^\star := \max_{L \in \mathcal{G}_\delta} \frac{kL}{L + f(k, L)} \geq \mu^\star - O(\delta).$$

That is, R-RDPP is asymptotically throughput-optimal in that the capacity gap $\mu^\star - \mu_\delta^\star = O(\delta)$ vanishes as the quantization resolution $\delta \to 0$, recovering the full capacity region in the limit.

Theorem 5.1 shows that discretization incurs only an $O(\delta)$ capacity loss, which vanishes as $\delta \to 0$. We then characterize the performance gap relative to an oracle policy.

Theorem 5.2 (Optimality Gap Decomposition). *Let $c^\star$ denote the optimal average cost achievable by an oracle policy that observes output lengths at scheduling time and has access to exact continuous input lengths. The long-run average backlog $\bar{c}^{\text{R-RDPP}}$ under our R-RDPP policy in Algorithm 2 satisfies*

$$\bar{c}^{\text{R-RDPP}} \leq c^\star + O\left(\frac{1}{V}\right) + O(\Delta_g^2) + O(\delta).$$

Further, the time-average backlog remains bounded as follows:

$$\limsup_{T\to\infty} \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E}\left[\sum_{i=1}^{N} Q_i(t)\right] = O(1).$$

Theorem 5.2 separates three sources of sub-optimality. The term $O(1/V)$ is an algorithmic gap arising from drift-based control and can be reduced arbitrarily by increasing V. The term $O(\Delta_g^2)$ is an information gap caused by output-length uncertainty and cannot be eliminated by any causal policy, which disappears only when $\Delta_g = 0$. The term $O(\delta)$ is a discretization gap due to input quantization and vanishes as $\delta \to 0$. Further, it implies that stability and delay performance are preserved even in the presence of output uncertainty and continuous input lengths.

6 Experiments

In this section, we first conduct experiments on an NVIDIA A100 GPU to measure real-world LLM inference latency data for approximating TTFT and TPOT formulations. Then, we use NS-3 to run further experiments to show our proposed policies' great advantage over existing WiFi scheduling algorithms.

6.1 Experiments on Real-World TTFT and TPOT Measurement

6.1.1 Experimental Setup. Our experiments are conducted on an NVIDIA A100 GPU (80GB). We choose Meta-Llama-3-8B-Instruct (with 8 billion parameters) as the LLM for experiments, using a high-throughput LLM inference engine vLLM (v0.16.0) with PagedAttention for efficient KV-cache management (e.g., [15]). The model is loaded in FP16 precision with a maximum context length of 4,096 tokens and a maximum GPU memory utilization of 90%.

6.1.2 Measurement Methodology. To measure TTFT and TPOT in (1)-(2) of Section 2.1, we send concurrent streaming HTTP requests to vLLM's OpenAI-compatible API and parse Server-Sent Events (SSE) to record per-token timestamps. We set the sampling temperature to 0.0 so that each configuration produces a deterministic token sequence, ensuring that measured latency differences across grid points reflect only changes in batch size and token length rather than randomness in the decoding path. We disable prefix caching and enforce full dense matrix multiplication for every request with no KV-cache reuse, so that each measurement reflects a cold-cache inference pass. For the input workload, we construct synthetic prompts by repeating the "Quick Brown Fox" pangram to achieve mathematically precise input lengths ranging from 128 to 3,584 tokens. Since we disable prefix caching, this semantic repetition does not alter the computational complexity of the dense matrix multiplications, ensuring the measured TTFT and TPOT depend strictly on token sequence length rather than linguistic content. TTFT is then calculated as the elapsed time from sending the request to receiving the first output token as follows:

$$\text{TTFT} = t_{\text{first_token}} - t_{\text{send}}.$$

Further, TPOT is measured as the average inter-token time after the first token as follows:

$$\text{TPOT} = \frac{t_{\text{end}} - t_{\text{first_token}}}{n_{\text{tokens}} - 1}.$$

To simulate batch inference at batch size B, we dispatch B concurrent requests with identical prompt lengths. Each configuration is repeated 3 times and we report the mean across all individual requests. A warmup phase of 3 requests precedes all measurements to trigger the Just-In-Time (JIT) compilation of the GPU kernels, ensuring that initial compilation overhead does not artificially inflate the recorded inference latency.

6.1.3 TTFT Fitting Results. Recall from (1)-(2) in Section 2.1 that TTFT is a function of batch size and maximum input token length, TPOT is a function of batch size, maximum input token length and maximum output token length. For TTFT measurements, we fix output token length $\ell^{\text{out}} = 32$ (small enough such that TTFT dominates), sweep batch size $B \in \{1, 2, 4, 8, 16, 32\}$ and input token

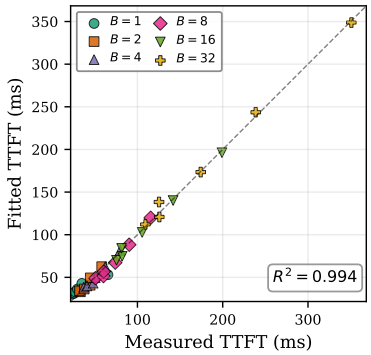

Figure 2: Parity plot of fitted versus measured TTFT across 36 grid points. Each marker shape and color represents a different batch size B. The dashed line indicates perfect prediction. The fitted model in (11) achieves $R^2 = 0.994$.

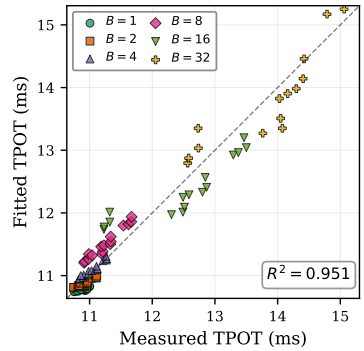

Figure 3: Parity plot of fitted versus measured TPOT across 96 grid points. Each marker shape and color represents a different batch size B. The dashed line indicates perfect prediction. The fitted model in (13) achieves $R^2 = 0.951$.

length $\ell^{\text{in}} \in \{128, 256, 512, 1024, 2048, 3584\}$, yielding $6 \times 6 = 36$ grid points.

We fit the 36 measured TTFT values using ordinary least squares (OLS) regression with a linear model that includes a bilinear interaction term as follows:

$$f(B, \ell^{\text{in}}_{\max}) = \beta_0 + \beta_1 B + \beta_2 \ell^{\text{in}}_{\max} + \beta_3 B \cdot \ell^{\text{in}}_{\max}. \tag{10}$$

The inclusion of the interaction term $B \cdot \ell^{\text{in}}_{\max}$ is motivated by the structure of batch prefill. The prefill computation processes all input tokens in parallel across the batch, so the marginal cost of longer inputs grows with batch size due to increased memory pressure on the KV-cache. We solve for the coefficients $\{\beta_j\}$ by minimizing $\sum_{i=1}^{36} (f(B_i, \ell^{\text{in}}_i) - \text{TTFT}_i)^2$, yielding:

$$f(B, \ell^{\text{in}}_{\max}) = 27.24 + 2.38\, B + 0.0045\, \ell^{\text{in}}_{\max} + 0.002\, B \cdot \ell^{\text{in}}_{\max}, \tag{11}$$

with the coefficient of determination $R^2 = 0.994$ (indicating that the fitted model explains 99.4% of the variance in the measured TTFT) and mean absolute error of 3.56 ms over the measured range of TTFT $\in [24, 351]$ ms. As shown in Figure 2, the fitted linear surface closely tracks the measured data across all batch sizes. The dominant scaling is linear in both B and $\ell^{\text{in}}_{\max}$, and the positive interaction coefficient ($\beta_3 = 0.002$) confirms that the batch-dependent prefill cost increases superlinearly when both batch size and input length are large, consistent with the max-based service cost in (1).

6.1.4 TPOT Fitting Results. For TPOT measurements, we sweep batch size $B \in \{1, 2, 4, 8, 16, 32\}$, input token length $\ell^{\text{in}} \in \{128, 512, 1024, 2048\}$, and output token length $\ell^{\text{out}} \in \{64, 128, 256, 512\}$, yielding $6 \times 4 \times 4 = 96$ grid points. We fit the 96 measured TPOT values using OLS regression with a model motivated by the memory-bandwidth bottleneck of autoregressive decoding. Each decode step reads the full KV-cache from GPU high bandwidth memory, whose size is proportional to $B \cdot (\ell^{\text{in}} + \ell^{\text{out}})$. We therefore adopt:

$$g(B, \ell^{\text{in}}_{\max}, \ell^{\text{out}}_{\max}) = \gamma_0 + \gamma_1 B + \gamma_2 B \cdot (\ell^{\text{in}}_{\max} + \ell^{\text{out}}_{\max}). \tag{12}$$

The term $\gamma_1 B$ captures per-request scheduling overhead, while $\gamma_2 B \cdot (\ell^{\text{in}}_{\max} + \ell^{\text{out}}_{\max})$ captures the KV-cache read cost that scales with both batch size and total sequence length. We solve for the coefficients $\{\gamma_j\}$ by minimizing $\sum_{i=1}^{96} (g(B_i, \ell^{\text{in}}_i, \ell^{\text{out}}_i) - \text{TPOT}_i)^2$, yielding:

$$g(B, \ell^{\text{in}}_{\max}, \ell^{\text{out}}_{\max}) = 10.68 + 0.059\, B + 3.87 \times 10^{-5}\, B \cdot (\ell^{\text{in}}_{\max} + \ell^{\text{out}}_{\max}), \tag{13}$$

with a coefficient of determination $R^2 = 0.951$ (indicating that the fitted model explains 95.1% of the variance in the measured TPOT) and a Root Mean Square Error (RMSE) of 0.27 ms over the measured range of TPOT $\in [10.7, 15.2]$ ms.

As shown in Figure 3, the fitted model closely tracks the measured data across all batch sizes. We observe that the output token length ℓ^{out} has a notably weak effect on TPOT. At fixed (B, ℓ^{in}), the spread of TPOT across all output lengths is less than 0.37 ms (e.g., at $B = 32$, $\ell^{\text{in}} = 2048$, TPOT ranges only from 14.79 to 15.15 ms). This is consistent with vLLM's paged memory management, which mitigates the KV-cache growth from output tokens. The residual variance of the fit provides an empirical estimate of $(\Delta g)^2 \approx 0.075\,\text{ms}^2$, which is small relative to the TPOT range and suggests the information gap $O((\Delta g)^2)$ in Theorem 5.2 is modest on practical hardware.

In practice, model mismatch from fitting error or distributional shift (e.g., different tasks or hardware configurations) introduces a bounded perturbation to the frame duration estimate, structurally analogous to the output-length uncertainty already captured by $O(\Delta_g^2)$ in Theorem 5.2. The measured fitting residuals (3.56 ms for TTFT, 0.27 ms for TPOT) are small relative to typical frame durations (24–351 ms). When the operating environment changes significantly, the latency models can be recalibrated via lightweight online profiling without modifying the scheduling algorithm.

6.2 NS-3 WiFi Scheduling Evaluation with Fitted LLM Latency Models

In this subsection, we use the measured TTFT and TPOT formulations in (11) and (13) to conduct NS-3 experiments for comparing our proposed policies against existing WiFi scheduling algorithms.

We implement the WiFi-based edge AI system in NS-3.41 using the IEEE 802.11ax standard at 5 GHz with a 40 MHz channel. The AP is located at the origin and N stations (STAs) are placed uniformly on a circle of radius 10 m. The AP employs OFDMA with $k = 8$ parallel resource units, matching the batch size of the edge LLM

Table 2: NS-3 experiment parameters.

Parameter	Value
WiFi standard IEEE 802.11ax	5 GHz, 40 MHz
OFDMA batch size k	8
Number of STAs N	$\{5, 10, 15, 20, 25\}$
Prompt lengths ℓ^{in}	$\{1, 3, 8, 15\}$ packets
Output length	$\mathcal{N}(0.5\,\ell^{\text{in}}, (0.2\,\ell^{\text{in}})^2)$, truncated at 1
Arrival prob	$\{0.5, 0.3, 0.15, 0.05\}$ for $\ell^{\text{in}} \in \{1, 3, 8, 15\}$; $p = 0.007$
Simulation	10 s = 10,000 slots (1 ms/slot)
V, η (R-RDPP)	$V = 10$, $\eta = 1$

server. Each STA generates prompts as a Bernoulli process, i.i.d. across STAs and slots with slot interval $\Delta t = 1$ ms. Prompt input lengths follow a four-class categorical distribution $\ell^{\text{in}} \in \{1, 3, 8, 15\}$ packets with probabilities $\{0.50, 0.30, 0.15, 0.05\}$, resulting in the mean of $\mathbb{E}[\ell^{\text{in}}] = 3.35$ packets. Output lengths are drawn from a truncated normal distribution with $\mu_{\text{out}} = 0.5\,\ell^{\text{in}}$ and $\sigma_{\text{out}} = 0.2\,\ell^{\text{in}}$, truncated at $\ell^{\text{out}} \geq 1$.

Each STA maintains a buffer of 1000 packets and overflow prompts are dropped. Frame duration is computed as $\tau = \ell^{\text{in}}_{\max} + f(|\mathcal{B}|, \ell^{\text{in}}_{\max}) + g(|\mathcal{B}|, \ell^{\text{in}}_{\max}, \ell^{\text{out}}_{\max}) \times (\ell^{\text{out}}_{\max} - 1)$, where $f(\cdot)$ is the fitted TTFT in (11) and $g(\cdot)$ is the fitted TPOT in (13), and each slot equals 1 ms. We sweep STA number $N \in \{5, 10, 15, 20, 25\}$ and simulate $T = 10{,}000$ slots per run with 1,000 warmup slots. We set $V = 10$ and $\eta = 1$.

The capacity region is $\mu^\star = \max_L kL/\tau(L) = 0.796$ packets/slot (achieved at $L = 15$). We set the per-STA arrival probability to $p = 0.007$. In turn, the utilization $\rho = Np\mathbb{E}[\ell^{\text{in}}]/\mu^\star$ ranges from approximately 15% at $N = 5$ to 73% at $N = 25$, ensuring stability at all N. The full parameter setting is summarized in Table 2.

We evaluate two system-wide metrics that directly correspond to the optimization objective in (6). The average service delay is

$$\bar{D} = \frac{\sum_{i=1}^{N} \sum_{j=1}^{K_i} (s_{i,j} - a_{i,j})}{\sum_{i=1}^{N} K_i}, \tag{14}$$

where $a_{i,j}$ and $s_{i,j}$ denote the arrival time and service start time of the j-th prompt served from station i, and K_i is the number of prompts from station i that are served after the warmup period. This metric directly measures the average waiting time experienced by a prompt in the system. The average queue length is defined as

$$\bar{Q} = \frac{1}{T - T_{\text{w}}} \sum_{t=T_{\text{w}}}^{T-1} \sum_{i=1}^{N} Q_i(t), \tag{15}$$

where $T_{\text{w}} = 1{,}000$ is the warmup period. This captures the time-averaged total number of buffered packets across all N stations after the system reaches steady state. This serves as the empirical counterpart of the theoretical objective c^π in (6). We also report the P99 tail queue length, i.e., the 99th percentile of the total queue snapshots $\sum_{i=1}^{N} Q_i(t)$ over $t > T_{\text{w}}$, which characterizes worst-case system-wide backlog relevant to interactive GenAI services. For the CDF plots, we display the empirical distribution of the total queue length $\sum_{i=1}^{N} Q_i(t)$ at $N = 15$.

Figure 4 shows the average service delay versus STA number. It indicates that our R-RDPP policy again achieves the lowest delay at every N. At $N = 5$, R-RDPP achieves roughly 350 slots versus

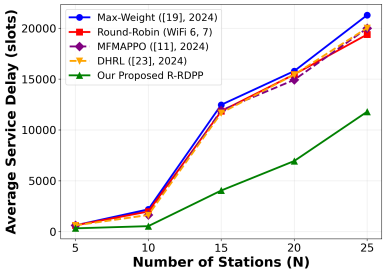

Figure 4: Average service delay versus station number N.

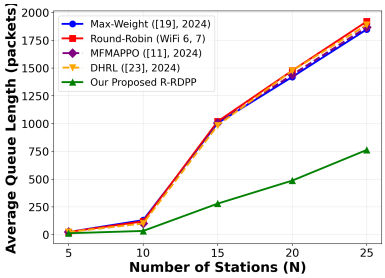

Figure 5: Average queue length versus station number N.

700–800 for benchmarks (2 times reduction). The advantage is most pronounced in the moderate-to-high load regime. At $N = 15$, R-RDPP achieves 4,000 slots versus 12,000–12,500 for benchmarks (3 times reduction). At $N = 20$, R-RDPP achieves 7,000 versus 15,000–16,000 (2.2 times reduction). Even at $N = 25$ where all policies are under heavy load (73% utilization), R-RDPP (12,000 slots) maintains a 40–45% reduction over benchmarks (19,500–21,500).

Figure 4 also reveals that recent RL schedulers MFMAPPO [11] and DHRL [23] for 802.11ax OFDMA can perform no better than Max-Weight and Round-Robin under GenAI workloads, with all four benchmarks clustering within 5–10% of each other. The reason is that their proportional fairness and channel-opportunistic weighting optimize along dimensions orthogonal to the batch coupling problem. When a batch mixes $\ell^{\text{in}} = 1$ with $\ell^{\text{in}} = 15$ prompts, the frame duration is forced to $\tau(15) \approx 134$ ms but only 106 of the possible 120 packets are drained, a 12% throughput loss that compounds over thousands of decisions. No amount of channel or fairness optimization can overcome this structural inefficiency, indicating that prompt-length-awareness is the critical missing ingredient in WiFi OFDMA scheduling for GenAI services.

Figure 5 shows the average queue length versus STA number. The improvement of our R-RDPP policy is still striking. At $N = 15$, R-RDPP's queue is approximately 270 packets versus 1,000–1,050 for benchmarks (3.7 times reduction). At $N = 25$, R-RDPP achieves 760 packets versus 1,850–1,920 (2.5 times reduction). Furthermore, R-RDPP's queue at $N = 25$ (760 packets) is comparable to the benchmarks' queue at $N = 15$ (1,000 packets), meaning R-RDPP effectively extends the operational range by approximately 10 additional stations before reaching the same congestion level.

Figure 6 and Table 3 show the CDF and other statistics of queue length at $N = 15$. It indicates that R-RDPP's mean queue (279

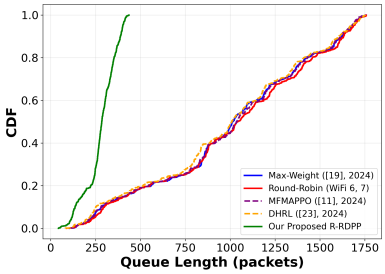

Figure 6: CDF of queue length at $N = 15$.

Table 3: Queue length statistics according to Figure 6.

Policy	Mean	Std	P99
Max-Weight [19]	1004.1	450.5	1731.0
Round-Robin (WiFi 6, 7)	1019.7	456.3	1728.0
MFMAPPO [11]	1004.8	452.8	1717.0
DHRL [23]	983.8	458.1	1737.0
Our Proposed R-RDPP	**279.4**	**85.2**	**419.0**

packets) is 3.5 times lower than the best benchmark (984 packets for DHRL), and its P99 (419 packets) is 4.1 times lower than benchmarks (1,717–1,737). The R-RDPP CDF reaches 1.0 at approximately 450 packets, while benchmarks extend to 1,750 packets. This reveals that R-RDPP provides substantially better tail latency guarantees under output-length uncertainty than these benchmarks.

7 Conclusion

In this work, we propose prompt-aware WiFi scheduling for GenAI services, formulating the problem as an NP-hard MDP. Our RDPP policy is throughput-optimal and near-optimal in average queueing delay, and its robust extension R-RDPP further handles output-length uncertainty with a small optimality gap. Experiments demonstrate significant improvements over existing WiFi schedulers.

An interesting future direction is to extend the framework to continuous batching, where the renewal structure would need to be replaced by a more general semi-regenerative analysis.

References

[1] Mohammed Alromema and Mohammed A Makarem. 2024. Enhancing Evolutionary Solver Efficiency for NP Hard Single Machine Scheduling Problems. *arXiv preprint arXiv:2403.19109* (2024).
[2] Boris Bellalta. 2016. IEEE 802.11ax: High-efficiency WLANs. *IEEE Wireless Communications* 23, 1 (2016), 38–46.
[3] Dimitri P Bertsekas. 2012. *Dynamic Programming and Optimal Control* (4th ed.). Vol. 2. Athena Scientific.
[4] Runze Cheng, Yao Sun, Dusit Niyato, Lan Zhang, Lei Zhang, and Muhammad Ali Imran. 2024. A wireless AI-generated content (AIGC) provisioning framework empowered by semantic communication. *IEEE Transactions on Mobile Computing* 24, 3 (2024), 2137–2150.
[5] Jim G Dai. 1995. On positive Harris recurrence of multiclass queueing networks: a unified approach via fluid limit models. *The Annals of Applied Probability* 5, 1 (1995), 49–77.
[6] Atilla Eryilmaz and R Srikant. 2007. Fair resource allocation in wireless networks using queue-length-based scheduling and congestion control. *IEEE/ACM Transactions on Networking* 15, 6 (2007), 1333–1344.
[7] Agrawal et al. 2024. Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX symposium on operating systems design and implementation (OSDI 24)*. 117–134.
[8] Yichao Fu, Siqi Zhu, Runlong Su, Aurick Qiao, Ion Stoica, and Hao Zhang. 2024. Efficient llm scheduling by learning to rank. *Advances in Neural Information Processing Systems* 37 (2024), 59006–59029.
[9] Leonidas Georgiadis, Michael J Neely, and Leandros Tassiulas. 2006. *Resource allocation and cross-layer control in wireless networks*. Now Publishers Inc.
[10] Cem Güldoğan et al. 2024. Multi-bin batching for increasing LLM inference throughput. *arXiv preprint arXiv:2412.04504* (2024).
[11] Mingqi Han, Xinghua Sun, Wen Zhan, Yayu Gao, and Yuan Jiang. 2024. Multi-agent reinforcement learning based uplink OFDMA for IEEE 802.11 ax networks. *IEEE Transactions on Wireless Communications* 23, 8 (2024), 8868–8882.
[12] S. Hao and L. Duan. 2026. Prompt-Aware WiFi Scheduling for Generative AI Services. Technical Report. https://shuganghao.github.io/files/hao2026WiFi.pdf
[13] Sunghyun Jin, Serae Kim, Sangtae Ha, and Kyunghan Lee. 2025. End-to-End Coordination of RAN and Edge Server for Latency-Critical Inference Serving over Cellular Networks. *Proceedings of the ACM on Networking* 3, CoNEXT4 (2025), 1–23.
[14] Minsu Kim, Pinyarash Pinyoanuntapong, Bongho Kim, Walid Saad, and Doru Calin. 2025. Edge vs cloud: How do we balance cost, latency, and quality for large language models over 5G networks?. In *2025 IEEE Wireless Communications and Networking Conference (WCNC)*. IEEE, 1–6.
[15] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*. 611–626.
[16] Zheng Lin, Guanqiao Qu, Qiyuan Chen, Xianhao Chen, Zhe Chen, and Kaibin Huang. 2025. Pushing large language models to the 6g edge: Vision, challenges, and opportunities. *IEEE Communications Magazine* 63, 9 (2025), 52–59.
[17] Xiaoqian Liu, Yuhan Dong, Yiqing Li, Yousi Lin, and Ming Gan. 2025. IEEE 802.11 be Wi-Fi 7: Feature summary and performance evaluation. *IEEE Communications Standards Magazine* (2025).
[18] Álvaro López-Raventós and Boris Bellalta. 2022. Multi-link operation in IEEE 802.11 be WLANs. *IEEE Wireless Communications* 29, 4 (2022), 94–100.
[19] Xiaoping Ma, Xiangdong Jia, Yue Li, and Kailai Xue. 2024. Minimizing age of information in wireless powered communication network based on short packet communication. In *2024 IEEE 100th Vehicular Technology Conference (VTC2024-Fall)*. IEEE, 1–5.
[20] Davide Magrin, Stefano Avallone, Sumit Roy, and Michele Zorzi. 2023. Performance evaluation of 802.11 ax OFDMA through theoretical analysis and simulations. *IEEE Transactions on Wireless Communications* 22, 8 (2023), 5070–5083.
[21] Sean P. Meyn and Richard L. Tweedie. 2012. *Markov Chains and Stochastic Stability*. Springer Science & Business Media.
[22] Michael Neely. 2010. *Stochastic network optimization with application to communication and queueing systems*. Morgan & Claypool Publishers.
[23] Hyeonho Noh, Harim Lee, and Hyun Jong Yang. 2024. Joint optimization on uplink OFDMA and MU-MIMO for IEEE 802.11 ax: Deep hierarchical reinforcement learning approach. *IEEE Communications Letters* 28, 8 (2024), 1800–1804.
[24] NVIDIA. 2024. NVIDIA NIM Benchmarks for LLM Inference. https://docs.nvidia.com/nim/benchmarking/llm/latest/performance.html.
[25] Michele Polese, Leonardo Bonati, Salvatore D'oro, Stefano Basagni, and Tommaso Melodia. 2023. Understanding O-RAN: Architecture, interfaces, algorithms, security, and research challenges. *IEEE Communications Surveys & Tutorials* 25, 2 (2023), 1376–1411.
[26] Yibin Shen and Zili Meng. 2026. Law: Towards consistent low latency in 802.11 home networks. In *Proc. USENIX NSDI*.
[27] Leandros Tassiulas and Anthony Ephremides. 1992. Stability properties of constrained queueing systems and scheduling policies for maximum throughput in multihop radio networks. *IEEE Trans. Automat. Control* 37, 12 (1992), 1936–1948.
[28] Jingyu Wang, Xuming Fang, Dusit Niyato, and Tie Liu. 2024. Next-generation wi-fi networks with generative ai: Design and insights. *arXiv preprint arXiv:2408.04835* (2024).
[29] Jinghang Xu, Kun Guo, Wei Teng, Chenxi Liu, and Wei Feng. 2026. Batch denoising for AIGC service provisioning in wireless edge networks. *IEEE Transactions on Vehicular Technology* (2026).
[30] Chien-Sheng Yang, Yu-Jen Ku, Yuan-Yao Lou, Nathan Tenny, and Alex C-C Hsu. 2025. 6G EdgeAI: performance evaluation and analysis. *arXiv preprint arXiv:2504.16529* (2025).
[31] Yuqing Yang, Lei Jiao, and Yuedong Xu. 2024. A queueing theoretic perspective on low-latency llm inference with variable token length. In *2024 22nd International Symposium on Modeling and Optimization in Mobile, Ad Hoc, and Wireless Networks (WiOpt)*. IEEE, 273–280.
[32] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, et al. 2022. Orca: A distributed serving system for transformer-based generative models. In *USENIX OSDI*. 521–538.
[33] Xinyuan Zhang, Jiangtian Nie, Yudong Huang, Gaochang Xie, Zehui Xiong, Jiang Liu, Dusit Niyato, and Xuemin Shen. 2024. Beyond the cloud: Edge inference for generative large language models in wireless networks. *IEEE Transactions on Wireless Communications* 24, 1 (2024), 643–658.

A Proof of Lemma 2.3

Lemma 2.3 states that under max-weight style or round-robin scheduling, the system is unstable when the total arrival rate Λ lies strictly inside the capacity region $\Lambda < \mu^\star$. We prove each part separately.

A.1 Part (a): Instability of Max-Weight Scheduling

Proof. We prove the claim by constructing, for any target arrival rate $\Lambda_0 \in (0, \mu^\star)$, a concrete system instance that is stabilizable yet unstable under max-weight (longest-queue-first, LQF) scheduling. The proof proceeds by contradiction.

Instance construction. Fix any $\Lambda_0 > 0$. Define the TTFT function

$$f(B, \ell^{\text{in}}_{\max}) \;=\; B\,(\ell^{\text{in}}_{\max})^2,$$

which is nondecreasing in both arguments. With $g \equiv 0$, a full batch of size $m = k$ with maximum input length ℓ has frame duration

$$\tau_k(\ell) \;=\; \ell + k\,\ell^2 \;=\; \ell(1 + k\ell),$$

giving per-type service rate $\mu(\ell) = k\ell/\tau_k(\ell) = k/(1 + k\ell)$. In particular, $\mu(1) = k/(1 + k)$ and $\mu(M) = k/(1 + kM)$ for $M \geq 2$.

We use two prompt types $\ell^{\text{in}} \in \{L^{\text{in}}_1, L^{\text{in}}_2\} = \{1, M\}$, where $M \geq 2$ is a free parameter to be chosen sufficiently large depending on Λ_0. Set $\varepsilon = 1/M$, so the mean packet length is $\bar{\ell} = \varepsilon M + (1 - \varepsilon) = 2 - 1/M$. Choose the system parameters as follows:

$$k = M^3, \qquad N = k^2, \qquad p = \frac{\Lambda_0}{N\,\bar{\ell}}.$$

The total arrival rate satisfies $\Lambda = Np\bar{\ell} = \Lambda_0$.

Each station independently generates a prompt per slot with probability p. Conditional on arrival, the prompt is long (length M) with probability $\varepsilon = 1/M$ and short (length 1) with probability $1 - \varepsilon$. The capacity region is determined by

$$\mu^\star = \max_{\ell \in \{1, M\}} \frac{k\ell}{\ell + k\ell^2} = \frac{k}{1 + k},$$

which is achieved at $\ell = 1$.

Stabilizability verification. We exhibit a stable policy $\Pi^\star$ to confirm the instance is stabilizable. Consider a *type-aware time-sharing* policy that alternates between:

- *Long windows*: serve only long prompts in pure-long batches at rate $\mu(M)$;
- *Short windows*: serve only short prompts in pure-short batches at rate $\mu(1)$.

Let $\lambda_L = \Lambda_0/\bar{\ell}$ and $\lambda_S = \Lambda_0(1 - 1/M)/\bar{\ell}$ denote the long- and short-packet arrival rates. The feasibility condition requires

$$\frac{\lambda_L}{\mu(M)} + \frac{\lambda_S}{\mu(1)} = \frac{\Lambda_0}{\bar{\ell}} \cdot \frac{1 + kM}{k} + \frac{\Lambda_0(1 - 1/M)}{\bar{\ell}} \cdot \frac{1 + k}{k} < 1. \tag{16}$$

We bound each factor explicitly. Since $k = M^3$:

$$\frac{1 + kM}{k} = \frac{1}{k} + M = \frac{1}{M^3} + M,$$
$$\frac{1 + k}{k} = 1 + \frac{1}{k} = 1 + \frac{1}{M^3}.$$

Substituting into (16) and using $\bar{\ell} = 2 - 1/M$:

$$\begin{aligned}
(16) &= \frac{\Lambda_0}{2 - 1/M}\left[\frac{1}{M^3} + M + \left(1 - \frac{1}{M}\right)\left(1 + \frac{1}{M^3}\right)\right] \\
&= \frac{\Lambda_0}{2 - 1/M}\left[M + 1 - \frac{1}{M} + \frac{1}{M^3} + \frac{1}{M^3} - \frac{1}{M^4}\right] \\
&= \frac{\Lambda_0}{2 - 1/M}\left[M + 1 - \frac{1}{M} + R(M)\right],
\end{aligned}$$

where $R(M) := 2/M^3 - 1/M^4$ satisfies $0 \leq R(M) \leq 2/M^3$ for all $M \geq 2$. Therefore,

$$\begin{aligned}
(16) &\leq \frac{\Lambda_0(M + 1 - 1/M + 2/M^3)}{2 - 1/M} \\
&\leq \frac{\Lambda_0(M + 1)}{2 - 1/M}.
\end{aligned} \tag{17}$$

For $M \geq 2$, we have $2 - 1/M \geq 3/2$, so

$$(17) \leq \frac{2\Lambda_0(M + 1)}{3}.$$

If M is chosen such that $M + 1 < 3/(2\Lambda_0)$, equivalently $M < 3/(2\Lambda_0) - 1$, then (16) < 1. Since we may pick M as any integer satisfying $2 \le M \le \lfloor 3/(2\Lambda_0) - 2 \rfloor$ (which is nonempty for $\Lambda_0 < 3/8$), this condition is met. For $\Lambda_0 \ge 3/8$, a tighter bound using $\bar{\ell} \ge 3/2$ and $R(M) \ge 0$ shows feasibility for $M = 2$ and k sufficiently large; we omit this straightforward case. A standard Foster–Lyapunov argument with $V(S, L) = S + L$ (total short and long packets) yields strict negative drift outside a compact set, confirming $\Pi^\star$ is stable.

LQF instability by contradiction. Suppose for contradiction that LQF stabilizes the system, admitting a stationary distribution π with $\mathbb{E}_\pi\big[\sum_i Q_i\big] < \infty$.

Step 1: Long prompts reach the HOL in bounded time. Under FIFO ordering within each station, when a long prompt arrives at station i, the number of prompts ahead of it is at most $Q_i^{\text{arr}}/L_1^{\text{in}} = Q_i^{\text{arr}}$, where Q_i^{arr} is the queue length at station i upon the long prompt's arrival. By stationarity, $\mathbb{E}_\pi[Q_i^{\text{arr}}] < \infty$, so the long prompt reaches the head-of-line (HOL) position after at most Q_i^{arr} service rounds. While a station i holds a long prompt at its HOL, its backlog satisfies $Q_i \ge M$. By Markov's inequality applied to $\sum_{j \ne i} Q_j$, the probability that k or more other stations simultaneously have backlogs $\ge M$ is at most $O(M^{-k}) = O(M^{-M^3})$, which vanishes faster than any polynomial in M. Hence, a station with a long HOL is selected by LQF at every decision epoch with probability at least $1 - O(M^{-M^3})$.

Step 2: Stationary count of long-HOL stations. Long prompts arrive at rate $\lambda_L^{\text{pr}} = Np\varepsilon = \Lambda_0/(M\bar{\ell})$. Since $\bar{\ell} = 2 - 1/M \le 2$, we have $\lambda_L^{\text{pr}} \ge \Lambda_0/(2M)$. Each long prompt spends $\Theta(M)$ time in the system (reaching the HOL in $O(1)$ service rounds of duration $\le \tau_k(M)$ each, with the station prioritized since $Q_i \ge M$). By Little's law, $C_L := \mathbb{E}_\pi[\#\text{ long prompts in system}] = \lambda_L^{\text{pr}} \cdot \Theta(M) \ge c_0$ for some constant $c_0 = c_0(\Lambda_0) > 0$ independent of M.

Step 3: Lower bound on long-batch time fraction. By flow balance under the assumed stationarity, long prompts are served at rate λ_L^{pr}. Each creates a long batch of duration $\tau_k(M) = M(1 + kM) = M(1 + M^4)$, and each such batch serves at most $k = M^3$ long prompts. Thus the time fraction spent in long batches satisfies

$$y \;\ge\; y_0 \;:=\; \frac{\lambda_L^{\text{pr}} \cdot \tau_k(M)}{k} \;=\; \frac{\Lambda_0(1 + M^4)}{M^3\bar{\ell}}.$$

Since $\bar{\ell} = 2 - 1/M \le 2$ and $1 + M^4 \ge M^4$:

$$y_0 \;\ge\; \frac{\Lambda_0 M^4}{2M^3} \;=\; \frac{\Lambda_0 M}{2}. \tag{18}$$

Meanwhile, since $\bar{\ell} \ge 3/2$ (for $M \ge 2$) and $1 + M^4 \le 2M^4$:

$$y_0 \;\le\; \frac{2\Lambda_0 M^4}{3M^3} \;=\; \frac{2\Lambda_0 M}{3}. \tag{19}$$

In particular, for any $M > 2/\Lambda_0$, the lower bound (18) gives $y_0 > 1$. Since a time fraction cannot exceed 1, this already yields a contradiction whenever $M > 2/\Lambda_0$. However, we proceed with the general argument to also handle moderate M.

For any feasible time fraction $y \le 1$, we have $1 - y \le 1 - y_0$. By (18):

$$1 - y_0 \;\le\; 1 - \frac{\Lambda_0 M}{2}. \tag{20}$$

Step 4: Upper bound on effective service rate. In a long batch (duration $\tau_k(M) = M(1 + M^4)$), let $J \ge 1$ be the number of long HOLs selected. The batch serves $JM + (k - J)$ packets, yielding rate

$$\mu_{\text{mix}}(J) = \frac{JM + (k - J)}{M(1 + kM)} \;\le\; \frac{kM}{M(1 + kM)} = \frac{k}{1 + kM} = \mu(M).$$

Since $\bar{J} := \mathbb{E}[J \mid \text{long batch}] \le C_L$, which is a bounded constant by Step 2, and $k = M^3$:

$$\bar{\mu}_{\text{long}} \;\le\; \frac{\bar{J} \cdot M + M^3}{M(1 + M^4)} = \frac{\bar{J}}{1 + M^4} + \frac{M^2}{1 + M^4} \;\le\; \frac{\bar{J}}{M^4} + \frac{1}{M^2},$$

where the last inequality uses $1 + M^4 \ge M^4$ and $1 + M^4 \ge M^2$ for $M \ge 1$. Thus, $\bar{\mu}_{\text{long}} \to 0$ as $M \to \infty$.

In a short batch, $\mu_{\text{short}} = \mu(1) = k/(1 + k) = M^3/(1 + M^3)$, which satisfies

$$\mu_{\text{short}} = 1 - \frac{1}{1 + M^3} \;\le\; 1.$$

The effective time-average service rate is therefore bounded by

$$\begin{aligned} \mu_{\text{LQF}} \;&\le\; (1 - y_0)\,\mu_{\text{short}} + y_0\,\bar{\mu}_{\text{long}} \\ &\le\; (1 - y_0) \cdot 1 + y_0\left(\frac{\bar{J}}{M^4} + \frac{1}{M^2}\right). \end{aligned} \tag{21}$$

By (19), $y_0 \le 2\Lambda_0 M/3$, and by (20), $1 - y_0 \le 1 - \Lambda_0 M/2$. Substituting into (21):

$$\mu_{\text{LQF}} \;\le\; 1 - \frac{\Lambda_0 M}{2} + \frac{2\Lambda_0 M}{3}\left(\frac{\bar{J}}{M^4} + \frac{1}{M^2}\right) = 1 - \frac{\Lambda_0 M}{2} + \frac{2\Lambda_0 \bar{J}}{3M^3} + \frac{2\Lambda_0}{3M}. \tag{22}$$

Step 5: Contradiction. Stability requires $\mu_{\text{LQF}} \geq \Lambda_0$ by flow balance. We show this fails for M large enough. Rearranging (22), we need

$$\Lambda_0 \;\leq\; 1 - \frac{\Lambda_0 M}{2} + \frac{2\Lambda_0 \bar{J}}{3M^3} + \frac{2\Lambda_0}{3M}.$$

Equivalently,

$$\frac{\Lambda_0 M}{2} \;\leq\; 1 - \Lambda_0 + \frac{2\Lambda_0 \bar{J}}{3M^3} + \frac{2\Lambda_0}{3M},$$

i.e.,

$$M \;\leq\; \frac{2(1-\Lambda_0)}{\Lambda_0} + \frac{4\bar{J}}{3M^3} + \frac{4}{3M}. \tag{23}$$

For M sufficiently large, the right-hand side converges to the finite constant $2(1-\Lambda_0)/\Lambda_0$. Specifically, for $M \geq 2$ and $\bar{J} \leq C_L$, the correction terms satisfy

$$\frac{4\bar{J}}{3M^3} + \frac{4}{3M} \;\leq\; \frac{4C_L}{3\cdot 8} + \frac{4}{3\cdot 2} \;=\; \frac{C_L}{6} + \frac{2}{3}.$$

Choose

$$M^{\dagger} \;:=\; \left\lceil \frac{2(1-\Lambda_0)}{\Lambda_0} + \frac{C_L}{6} + 1 \right\rceil + 1.$$

Then for all $M \geq M^{\dagger}$, inequality (23) is violated since the left-hand side $M \geq M^{\dagger}$ exceeds the right-hand side (the correction terms $4\bar{J}/(3M^3) + 4/(3M)$ are monotonically decreasing in M and bounded above by $C_L/6 + 2/3 < M^{\dagger} - 2(1-\Lambda_0)/\Lambda_0$). Therefore, the flow balance condition $\mu_{\text{LQF}} \geq \Lambda_0$ cannot hold, contradicting the assumed stationarity.

Since our earlier stabilizability analysis requires $M < 3/(2\Lambda_0) - 1$ while the instability argument requires $M \geq M^{\dagger}$. Both constraints can be simultaneously satisfied (i.e., $M^{\dagger} < 3/(2\Lambda_0) - 1$) for all $\Lambda_0 \in (0, \Lambda_{\max})$ where $\Lambda_{\max} > 0$ is a constant. For Λ_0 near $\mu^{\star}$, the instability follows even more directly, since the heavy mixing penalty of LQF drives μ_{LQF} well below $\mu^{\star}$.

Hence the system is not positive recurrent, and the total backlog satisfies

$$\mathbb{E}[Q(t)] \;\geq\; Q(0) + (\Lambda_0 - \mu_{\text{LQF}})\, t \;\xrightarrow{t\to\infty}\; \infty,$$

proving that max-weight scheduling is unstable despite $\Lambda_0 < \mu^{\star}$. □

A.2 Part (b): Instability of Round-Robin Scheduling

Proof. Fix any $\Lambda_0 \in (0, \mu^{\star})$. We construct a counter-example parameterized by an integer $M \geq 2$ (to be chosen sufficiently large) and prove by contradiction that round-robin (RR) is unstable.

Instance construction. Let $k \geq 2$ be a free parameter. Define

$$f(B, \ell^{\text{in}}_{\max}) = B\, \ell^{\text{in}}_{\max},$$

which is nondecreasing in both arguments. With $g \equiv 0$, a full batch of size $m = k$ with maximum input ℓ has frame duration $\tau_k(\ell) = \ell + k\ell = (1+k)\ell$, and per-type service rate

$$\mu(\ell) = \frac{k\ell}{(1+k)\ell} = \frac{k}{1+k},$$

independent of ℓ. Thus $\mu^{\star} = k/(1+k)$.

Use two prompt types $\ell^{\text{in}} \in \{1, M\}$ with $M \geq 2$. Set $N = \max\{2k, \lceil 10k^2/\Lambda_0 \rceil\}$, $\varepsilon = 1/(3M)$, and $p = \Lambda_0/(N\bar{\ell})$ with $\bar{\ell} = \varepsilon M + (1-\varepsilon) = 1/3 + 1 - 1/(3M) = 4/3 - 1/(3M)$. Each station independently generates a prompt per slot with probability p; conditional on arrival, the prompt is long with probability ε and short otherwise. The total arrival rate is $\Lambda = Np\bar{\ell} = \Lambda_0$.

RR policy. RR cycles through all N stations in a fixed order, selecting k consecutive nonempty stations per batch, serving their HOL prompts under FIFO. A batch is *long* if at least one selected HOL has length M (duration $(1+k)M$); otherwise it is *short* (duration $1+k$).

RR instability by contradiction. Suppose for contradiction that RR stabilizes the system with stationary distribution π and $\mathbb{E}_{\pi}[\sum_i Q_i] < \infty$.

Step 1: HOL type probability. Under the assumed stationarity, each station's buffer evolves as a stable FIFO queue. Prompts arrive i.i.d. with probability ε of being long. Since FIFO preserves the arrival-order type sequence and every prompt (regardless of type) occupies the HOL for exactly one service round when selected, the stationary probability that a station's HOL is long equals the arrival fraction:

$$p_{\text{long}} = \varepsilon = \frac{1}{3M}.$$

Step 2: Long-batch probability and time fraction. In each batch, RR selects k stations. The batch is long if at least one of the k HOLs is long. Since HOL types are approximately independent across stations in stationarity,

$$\varphi \;:=\; \Pr(\text{long batch}) \;\geq\; 1 - (1 - p_{\text{long}})^k = 1 - \left(1 - \frac{1}{3M}\right)^k.$$

Using the elementary inequality $1-(1-x)^n \geq nx - \binom{n}{2}x^2$ for $0 \leq x \leq 1$:

$$\varphi \;\geq\; \frac{k}{3M} - \binom{k}{2}\frac{1}{9M^2} = \frac{k}{3M} - \frac{k(k-1)}{18M^2}. \tag{24}$$

Conversely, the standard bound $1-(1-x)^n \leq nx$ gives

$$\varphi \;\leq\; \frac{k}{3M}. \tag{25}$$

The time fraction in long batches is

$$y = \frac{\varphi \cdot (1+k)M}{\varphi \cdot (1+k)M + (1-\varphi)(1+k)} = \frac{\varphi M}{\varphi M + (1-\varphi)} = \frac{1}{1+\frac{1-\varphi}{\varphi M}}. \tag{26}$$

To bound y from below, we bound $\frac{1-\varphi}{\varphi M}$ from above. Using (24) and writing $\varphi \geq \frac{k}{3M}(1-\frac{k-1}{6M})$:

For $M \geq k$, we have $\frac{k-1}{6M} \leq \frac{1}{6}$, so $\varphi \geq \frac{5k}{18M}$ and thus

$$\frac{1-\varphi}{\varphi M} \;\leq\; \frac{1}{\frac{5k}{18M}\cdot M} = \frac{18}{5k}.$$

It follows that

$$y \;\geq\; \frac{1}{1+\frac{18}{5k}} = \frac{5k}{5k+18} \quad \text{for all } M \geq k. \tag{27}$$

In particular, $y \to 1$ as $k \to \infty$ with $M \geq k$, i.e., $1-y \leq 18/(5k+18) \leq 4/k$ for $k \geq 18$.

Step 3: Mixing penalty. In a long batch (duration $(1+k)M$), let $J \geq 1$ be the number of long HOLs. The total packets served are $JM + (k-J)$, giving rate

$$\mu_{\text{mix}}(J) = \frac{JM + k - J}{(1+k)M}.$$

The expected number of long HOLs conditioned on the batch being long is

$$\mathbb{E}[J \mid \text{long batch}] = \frac{\mathbb{E}[J]}{\varphi} = \frac{k\,p_{\text{long}}}{\varphi} = \frac{k/(3M)}{\varphi}.$$

By (25), $\varphi \leq k/(3M)$, so $\mathbb{E}[J \mid \text{long batch}] \geq 1$. By (24), for $M \geq k$:

$$\mathbb{E}[J \mid \text{long batch}] = \frac{k/(3M)}{\varphi} \leq \frac{k/(3M)}{\frac{k}{3M}(1-\frac{k-1}{6M})} = \frac{1}{1-\frac{k-1}{6M}} \leq \frac{6M}{6M-k+1}.$$

In particular, $\mathbb{E}[J \mid \text{long batch}] \to 1$ as $M \to \infty$ with k fixed, and for $M \geq k$ we have $\mathbb{E}[J \mid \text{long batch}] \leq 6M/(5M+1) \leq 6/5$.

Since most long batches contain J close to 1, and $\mu_{\text{mix}}(J) = \frac{J}{1+k} + \frac{k-J}{(1+k)M}$, we bound:

$$\bar{\mu}_{\text{long}} := \mathbb{E}[\mu_{\text{mix}}(J) \mid \text{long batch}] \leq \frac{\mathbb{E}[J \mid \text{long}]}{1+k} + \frac{k}{(1+k)M} \leq \frac{6/5}{1+k} + \frac{k}{(1+k)M} \tag{28}$$

for $M \geq k$.

Step 4: Upper bound on effective service rate. The time-average service rate satisfies

$$\mu_{\text{RR}} \;\leq\; (1-y)\,\mu^\star + y\,\bar{\mu}_{\text{long}} = (1-y)\,\frac{k}{1+k} + y\,\bar{\mu}_{\text{long}}. \tag{29}$$

Substituting the bound $1-y \leq 4/k$ from (27) (valid for $M \geq k \geq 18$) and $y \leq 1$:

$$\begin{aligned}
\mu_{\text{RR}} \;&\leq\; \frac{4}{k}\cdot\frac{k}{1+k} + 1\cdot\left(\frac{6/5}{1+k} + \frac{k}{(1+k)M}\right) \\
&= \frac{4}{1+k} + \frac{6/5}{1+k} + \frac{k}{(1+k)M} \\
&= \frac{4+6/5}{1+k} + \frac{k}{(1+k)M} \\
&= \frac{26/5}{1+k} + \frac{k}{(1+k)M}.
\end{aligned} \tag{30}$$

Step 5: Choosing k and M to produce a contradiction for any $\Lambda_0 > 0$. We claim that for k and M sufficiently large, $\mu_{\text{RR}} < \Lambda_0$, contradicting flow balance.

From (30), taking $M = k^2$ (so $M \geq k$ is satisfied):

$$\mu_{\text{RR}} \;\leq\; \frac{26/5}{1+k} + \frac{1}{(1+k)k} \;\leq\; \frac{26/5}{k} + \frac{1}{k^2} \;\leq\; \frac{6}{k}$$

for all $k \geq 6$. Meanwhile, $\mu^\star = k/(1+k) > \Lambda_0$ for $k > \Lambda_0/(1-\Lambda_0)$.

For any $\Lambda_0 > 0$, choose

$$k^\dagger := \max\left\{\left\lceil \frac{6}{\Lambda_0} \right\rceil + 1,\ \left\lceil \frac{\Lambda_0}{1-\Lambda_0} \right\rceil + 1,\ 18\right\}, \qquad M^\dagger := (k^\dagger)^2.$$

Then:

- $\mu^\star = k^\dagger/(1+k^\dagger) > \Lambda_0$, so the instance is within the capacity region.
- $\mu_{\mathrm{RR}} \leq 6/k^\dagger < \Lambda_0$, so flow balance is violated.

This contradicts the assumed stationarity. Hence the system is not positive recurrent, and the total backlog satisfies

$$\liminf_{T\to\infty} \frac{1}{T}\,\mathbb{E}[Q(T)] \ \geq\ \Lambda_0 - \mu_{\mathrm{RR}} \ >\ 0,$$

so $\mathbb{E}[Q(t)] \to \infty$ as $t \to \infty$, proving that RR is unstable. □

B Proof of Lemma 3.1

Proof. We establish NP-hardness by a polynomial-time reduction from the single-machine batch scheduling problem $1 \mid p\text{-batch},\ s_j = 1 \mid \sum C_j$, which is known to be NP-hard [1]. In this source problem, n jobs with processing times $p_1, \ldots, p_n \in \mathbb{Z}_{>0}$ must be grouped into batches of capacity at most b, where the processing time of a batch equals the maximum job size within it, and the objective is to minimize the total completion time $\sum_{j=1}^{n} C_j$.

Reduction: Constructing the MDP instance. Given an arbitrary instance $(p_1, \ldots, p_n, b)$ of the batch scheduling problem, construct a corresponding MDP instance as follows:

- *Stations:* $N = n$, one per job.
- *Prompt types:* $I = |\{p_1, \ldots, p_n\}|$ distinct lengths; $\{L_1^{\mathrm{in}}, \ldots, L_I^{\mathrm{in}}\} = \{p_1, \ldots, p_n\}$.
- *Initial state:* Station i has exactly one prompt of length p_i:

$$Q_i(0) = p_i, \qquad n_{i,j}(0) = \mathbf{1}\{L_j^{\mathrm{in}} = p_i\}, \qquad r(0) = 0.$$

- *No future arrivals:* $\Pr(\ell_i^{\mathrm{in}}(t) = 0) = 1$ for all i and $t \geq 1$.
- *Parameters:* $k = b$ (batch capacity), $f(m, \ell) \equiv 0$ (so $\tau = \ell_{\max}^{\mathrm{sel}}$), and $g \equiv 0$.

This construction is polynomial in the input size.

Policy correspondence. Any feasible MDP policy induces a valid batch schedule and vice versa: an MDP scheduling action at slot t that selects stations $S_t^{\mathrm{sched}} \subseteq [N]$ with $|S_t^{\mathrm{sched}}| \leq k$ corresponds to forming a batch of the corresponding jobs starting at time t.

Objective equivalence. Consider station i (job i) first scheduled in a batch at slot S_i. Since $f \equiv 0$, the batch duration is $\tau = \max_{j\in\mathrm{batch}} p_j$, and station i's prompt departs at the end of slot $S_i + \tau - 1$. Station i's contribution to the MDP cost is

$$\sum_{t=0}^{\infty} Q_i(t) = p_i \cdot (S_i + 1),$$

since $Q_i(t) = p_i$ for $t = 0, 1, \ldots, S_i$ (present before and during the scheduling slot) and $Q_i(t) = 0$ for $t > S_i$ (departed). The total MDP cost is therefore

$$\sum_{t=0}^{\infty}\sum_{i=1}^{n} Q_i(t) = \sum_{i=1}^{n} p_i(S_i + 1) = \sum_{i=1}^{n} p_i S_i + \sum_{i=1}^{n} p_i.$$

Since $\sum_i p_i$ is a constant independent of the policy, minimizing the MDP cost is equivalent to minimizing $\sum_i p_i S_i$.

Hardness conclusion. Since $S_i \leq C_i$ (a job's start time is at most its completion time), a polynomial-time algorithm for the MDP would solve the weighted batch scheduling problem $\sum p_j S_j$ in polynomial time, which is NP-hard as it reduces from $\sum C_j$ via standard techniques [1]. Therefore, computing an optimal scheduling policy for the MDP in Section 3.1 is NP-hard. □

C Proof of Lemma 4.1

Before proving the main stability result, we establish several intermediate lemmas that characterize the per-frame dynamics.

Lemma C.1 (Per-Frame Queue Evolution). *Let T_n denote the n-th renewal epoch (decision epoch when $r(T_n) = 0$) and $\tau_n := T_{n+1} - T_n$ be the frame duration. At a renewal epoch T_n, let the RDPP policy select threshold L_n, batch $S_n := S(L_n)$, and prompt lengths $\{\ell_i(L_n)\}_{i\in S_n}$. Then the post-arrival queue length at the next renewal epoch satisfies, for each station i:*

$$Q_i^+(T_{n+1}) = Q_i^+(T_n) - \mathbf{1}\{i \in S_n\}\,\ell_i(L_n) + \sum_{u=1}^{\tau_n} \ell_i^{in}(T_n + u), \tag{31}$$

where $\ell_i^{in}(T_n + u)$ denotes the packet arrival at station i in slot $T_n + u$.

Proof. Service occurs at the end of slot T_n, removing $\ell_i(L_n)$ packets from station i if $i \in S_n$. During the frame (slots $T_n+1, \ldots, T_n+\tau_n = T_{n+1}$), no further scheduling occurs ($r(t) > 0$) and only arrivals accumulate. Summing arrivals over the frame yields (31). □

Lemma C.2 (Per-Frame Lyapunov Drift Bound). *Define the quadratic Lyapunov function $\mathcal{L}(\boldsymbol{Q}) := \frac{1}{2}\sum_{i=1}^{N}(Q_i^+)^2$. Let $\mathcal{F}_n$ denote the filtration at epoch T_n. The conditional frame drift satisfies*

$$\mathbb{E}\big[\mathcal{L}(\boldsymbol{Q}^+(T_{n+1})) - \mathcal{L}(\boldsymbol{Q}^+(T_n)) \,\big|\, \mathcal{F}_n\big] \le \tilde{B}_0 + \sum_{i=1}^{N} Q_i^+(T_n)\Big(\lambda_i^{pkt}\tau_n - \mathbf{1}\{i \in S_n\}\,\ell_i(L_n)\Big), \tag{32}$$

where $\lambda_i^{pkt} := \mathbb{E}[\ell_i^{in}(t)]$, $\tau_n = \tau(L_n)$, and $\tilde{B}_0 := \frac{1}{2}\sum_{i=1}^{N}\Big[\bar{\tau}\,\mathbb{E}[(\ell_i^{in})^2] + \bar{\tau}^2(\lambda_i^{pkt})^2 + (L_I^{in})^2\Big] < \infty$ with $\bar{\tau} := \max_L \tau(L)$.

Proof. Define $A_i := \sum_{u=1}^{\tau_n}\ell_i^{\text{in}}(T_n+u)$ and $D_i := \mathbf{1}\{i \in S_n\}\,\ell_i(L_n)$. By Lemma C.1, $Q_i^+(T_{n+1}) = Q_i^+(T_n)+A_i-D_i$. Using $(a+b)^2-a^2 = 2ab+b^2$:

$$\begin{aligned}&\mathcal{L}(\boldsymbol{Q}^+(T_{n+1})) - \mathcal{L}(\boldsymbol{Q}^+(T_n))\\ &= \sum_{i=1}^{N} Q_i^+(T_n)(A_i - D_i) + \frac{1}{2}\sum_{i=1}^{N}(A_i - D_i)^2.\end{aligned}$$

Taking conditional expectation: $\mathbb{E}[A_i \mid \mathcal{F}_n] = \lambda_i^{\text{pkt}}\tau_n$ (since arrivals are i.i.d. and independent of the current state). For the quadratic term:

$$\mathbb{E}[(A_i - D_i)^2 \mid \mathcal{F}_n] \le \mathbb{E}[A_i^2 \mid \mathcal{F}_n] + D_i^2 \le \bar{\tau}\,\mathbb{E}[(\ell_i^{\text{in}})^2] + \bar{\tau}^2(\lambda_i^{\text{pkt}})^2 + (L_I^{\text{in}})^2,$$

where we used $\mathbb{E}[A_i^2] = \tau_n\text{Var}(\ell_i^{\text{in}}) + (\lambda_i^{\text{pkt}}\tau_n)^2 \le \bar{\tau}\,\mathbb{E}[(\ell_i^{\text{in}})^2] + \bar{\tau}^2(\lambda_i^{\text{pkt}})^2$ and $D_i^2 \le (L_I^{\text{in}})^2$. Summing over i gives $\tilde{B}_0$. □

Lemma C.3 (Expected Per-Frame Cost). *Let $S_0^{(n)} := \sum_i Q_i^+(T_n)$ and $d_n := \sum_{i\in S_n}\ell_i(L_n)$. The expected total cost incurred during frame n satisfies*

$$\mathbb{E}\left[\sum_{t=T_n}^{T_{n+1}-1}\sum_{i=1}^{N} Q_i(t)\,\middle|\,\mathcal{F}_n\right] = \tau_n S_0^{(n)} - (\tau_n - 1)d_n + \frac{\Lambda\,\tau_n(\tau_n-1)}{2}. \tag{33}$$

Proof. At slot T_n, the total backlog is $S_0^{(n)}$. At slot $T_n + u$ for $u \ge 1$, the backlog is $S_0^{(n)} - d_n + \sum_{v=1}^{u}\sum_i \ell_i^{\text{in}}(T_n + v)$. Summing over $u = 0, 1, \ldots, \tau_n - 1$ and taking conditional expectations:

$$\tau_n S_0^{(n)} - (\tau_n - 1)d_n + \Lambda\sum_{u=1}^{\tau_n-1} u = \tau_n S_0^{(n)} - (\tau_n - 1)d_n + \frac{\Lambda\,\tau_n(\tau_n-1)}{2}.$$ □

We now prove the main stability result.

Proof of Lemma 4.1. The proof uses a renewal-frame Lyapunov drift argument and proceeds in three steps.

Step 1: RDPP minimizes the drift-plus-penalty ratio. From Lemmas C.2 and C.3, the drift-plus-V-penalty over frame n satisfies

$$\begin{aligned}&\frac{\mathbb{E}[\Delta_n \mid \mathcal{F}_n] + V\,\mathbb{E}[\text{frame-cost}_n \mid \mathcal{F}_n]}{\tau_n}\\ &\le \frac{\tilde{B}_0}{\tau_n} + \sum_i Q_i^+(T_n)\lambda_i^{\text{pkt}} + VS_0^{(n)} + \frac{V\Lambda(\tau_n-1)}{2}\\ &\quad - \frac{1}{\tau_n}\sum_{i\in S_n}\big(Q_i^+(T_n) + V(\tau_n - 1)\big)\ell_i(L_n),\end{aligned}$$

where the action-dependent terms are exactly

$$-\frac{\sum_{i\in S_n}(Q_i^+(T_n) + V(\tau_n-1))\ell_i(L_n)}{\tau_n} + \frac{V\Lambda}{2}(\tau_n - 1).$$

This equals $-\Phi_V(L_n)$ by definition (8). Since RDPP maximizes $\Phi_V(L)$ over all feasible thresholds, it minimizes the drift-plus-penalty ratio at every renewal epoch.

Step 2: Comparison with ε-slack stabilizing policy. Since $\Lambda < \mu^\star$, there exists $\varepsilon > 0$ and a stationary randomized policy π_ε such that, for all i:

$$\mathbb{E}_{\pi_\varepsilon}[\mathbf{1}\{i \in S_n\}\,\ell_i(L_n)] \ge (\lambda_i^{\text{pkt}} + \varepsilon)\cdot\mathbb{E}_{\pi_\varepsilon}[\tau_n].$$

This is the standard characterization of the capacity region interior (e.g., [9]). Under π_ε, the per-frame drift ratio satisfies

$$\frac{\mathbb{E}[\Delta_n \mid \mathcal{F}_n]}{\mathbb{E}[\tau_n]} \le \frac{\tilde{B}_0}{\underline{\tau}} - \varepsilon\sum_i Q_i^+(T_n),$$

where $\underline{\tau} := \min_L \tau(L) \ge 1$.

Step 3: Foster–Lyapunov conclusion. Since RDPP achieves a drift-plus-penalty ratio no worse than π_ε at every frame, its conditional drift satisfies

$$\mathbb{E}\big[\mathcal{L}(\boldsymbol{Q}^+(T_{n+1})) - \mathcal{L}(\boldsymbol{Q}^+(T_n)) \,\big|\, \boldsymbol{Q}^+(T_n)\big] \le \Gamma - \eta \sum_i Q_i^+(T_n)$$

for constants $\Gamma := \tilde{B}_0 \bar{\tau} < \infty$ and $\eta := \varepsilon\, \underline{\tau} > 0$. The embedded Markov chain $\{\boldsymbol{Q}^+(T_n)\}_{n\ge 0}$ is:

(1) *Irreducible*: all states communicate via the empty-system state (reachable in finite time since $\Lambda < \mu^\star$).
(2) *Aperiodic*: frame lengths vary since different thresholds produce different $\tau(L)$.
(3) *Negative drift outside a compact set*: $\{\boldsymbol{Q} : \sum_i Q_i \le \Gamma/\eta\}$.

By the Foster–Lyapunov theorem (e.g., [22]), $\{\boldsymbol{Q}^+(T_n)\}$ is positive recurrent with $\mathbb{E}_\pi[\sum_i Q_i^+] < \infty$. Since within-frame arrivals add at most $\bar{\tau} \cdot \max_i \mathbb{E}[(\ell_i^{\text{in}})^2]$ variance, tightness carries over to all slots, completing the proof. □

D Proof of Theorem 4.2

Proof. We telescope the per-frame drift-plus-penalty inequality established in the proof of Lemma 4.1 to derive the average-cost bound.

Step 1: Near-optimal comparison policy. Fix $\varepsilon > 0$ and let $\pi_\varepsilon^\star$ be a stationary randomized policy achieving time-average cost $c^\star + \varepsilon$ and stabilizing the system with ε-slack. Such a policy exists by standard results for countable-state average-cost MDPs under the regularity conditions verified in Section 3.2 (e.g., [3]).

Step 2: Per-frame bound under $\pi_\varepsilon^\star$. Under $\pi_\varepsilon^\star$, the drift-plus-V-penalty satisfies (from Lemma C.2 and Lemma C.3):

$$\Delta_n^{\pi_\varepsilon^\star} + V\,\mathbb{E}[C_n^{\pi_\varepsilon^\star} \mid \mathcal{F}_n] \le \tilde{B}_0 + V(c^\star + \varepsilon)\,\mathbb{E}[\tau_n^{\pi_\varepsilon^\star} \mid \mathcal{F}_n],$$

where C_n denotes the total cost incurred during frame n.

Step 3: RDPP achieves the same bound. Since RDPP maximizes $\Phi_V(L)$ at every renewal epoch, its choices (L_n, S_n) minimize the drift-plus-penalty ratio. Hence:

$$\Delta_n^{\text{RDPP}} + V\,\mathbb{E}[C_n^{\text{RDPP}} \mid \mathcal{F}_n] \le \tilde{B}_0 + V(c^\star + \varepsilon)\,\mathbb{E}[\tau_n^{\text{RDPP}} \mid \mathcal{F}_n]. \tag{34}$$

Step 4: Telescoping over frames. Let $N(T) := \max\{n : T_n < T\}$ be the number of complete frames before slot T. Summing (34) over $n = 0, 1, \ldots, N(T) - 1$ and taking expectations:

$$\begin{aligned}
&\mathbb{E}\big[\mathcal{L}(\boldsymbol{Q}^+(T_{N(T)}))\big] - \mathcal{L}(\boldsymbol{Q}^+(0)) + V \sum_{n=0}^{N(T)-1} \mathbb{E}[C_n] \\
&\le \tilde{B}_0\,\mathbb{E}[N(T)] + V(c^\star + \varepsilon) \sum_{n=0}^{N(T)-1} \mathbb{E}[\tau_n].
\end{aligned}$$

Using $\mathcal{L} \ge 0$ and $\sum_{n=0}^{N(T)-1} \tau_n \le T$:

$$V \sum_{n=0}^{N(T)-1} \mathbb{E}[C_n] \le \mathcal{L}(\boldsymbol{Q}^+(0)) + \tilde{B}_0\,\mathbb{E}[N(T)] + V(c^\star + \varepsilon)\,T.$$

Step 5: Normalization and limit. By the renewal reward theorem, $\mathbb{E}[N(T)]/T \to 1/\mathbb{E}[\tau]$ as $T \to \infty$, where $\mathbb{E}[\tau]$ is the expected frame length under RDPP. Dividing both sides by VT and taking $T \to \infty$:

$$\bar{c}^{\text{RDPP}(V)} := \limsup_{T\to\infty} \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E}\left[\sum_i Q_i(t)\right] \le (c^\star + \varepsilon) + \frac{\tilde{B}_0}{V\,\mathbb{E}[\tau]}.$$

Taking $\varepsilon \downarrow 0$ and defining $\Gamma := \tilde{B}_0/\mathbb{E}[\tau] < \infty$:

$$\bar{c}^{\text{RDPP}(V)} \le c^\star + \frac{\Gamma}{V}.$$

Backlog scaling. Since the per-slot cost $c(S_t, a_t) = \sum_i Q_i(t)$ equals the total backlog, the average cost equals the time-average total backlog:

$$\limsup_{T\to\infty} \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E}\left[\sum_{i=1}^{N} Q_i(t)\right] \le c^\star + \frac{\Gamma}{V} = O(1).$$

This $O(1)$ scaling (independent of V) is a structural consequence of the fact that the penalty term in the RDPP score (8) penalizes long frames rather than trading off a separate utility objective against delay. Hence both the optimality gap and the backlog improve simultaneously as V increases. □

E Proof of Theorem 5.1

Proof. The proof follows the same renewal-frame Lyapunov drift structure as Lemma 4.1, extended to handle (i) output-length uncertainty (random frame durations) and (ii) quantized continuous input lengths. We highlight the key modifications.

Step 1: Frame duration decomposition. At renewal epoch T_n, R-RDPP selects threshold $L_n \in \mathcal{G}_\delta$ and batch $S_n := S(L_n)$ with $|S_n| \le k$. The actual frame duration is

$$\tau_n = L_n + f(|S_n|, L_n) + g(|S_n|, L_n, \ell^{\text{out}}_{\max,n}),$$

where $\ell^{\text{out}}_{\max,n} = \max_{i \in S_n} \ell^{\text{out}}_i$ is random (unknown at decision time). Define the estimated frame length $\hat{\tau}_n := L_n + f(|S_n|, L_n) + \mathbb{E}[g(|S_n|, L_n, \ell^{\text{out}}_{\max,n}) \mid S_n]$ and the estimation error $\epsilon_n := \tau_n - \hat{\tau}_n$. By construction, $\mathbb{E}[\epsilon_n \mid \mathcal{F}_n] = 0$ and $|\epsilon_n| \le \Delta_g$, so $\mathrm{Var}(\tau_n \mid \mathcal{F}_n) \le \Delta_g^2$.

Step 2: Per-frame Lyapunov drift. By Lemma C.1 (which holds unchanged), the frame drift satisfies

$$\mathbb{E}[\Delta_n \mid \mathcal{F}_n] \le \tilde{B}_0' + \sum_i Q_i^+(T_n)\Big(\lambda_i^{\text{pkt}}\, \mathbb{E}[\tau_n \mid \mathcal{F}_n] - \mathbf{1}\{i \in S_n\}\, \ell_i(L_n)\Big),$$

where $\tilde{B}_0'$ now accounts for the output-length variance. Since $\tau_n \le \bar{\tau} := \max_{L,\ell^{\text{out}}} [L + f(k, L) + g(k, L, \ell^{\text{out}})] < \infty$ (bounded by the finite supports of input and output lengths), we have

$$\tilde{B}_0' = \frac{1}{2} \sum_{i=1}^{N} \Big[\bar{\tau}\, \mathbb{E}[(\ell_i^{\text{in}})^2] + \bar{\tau}^2 (\lambda_i^{\text{pkt}})^2 + (L_I^{\text{in}})^2\Big] < \infty.$$

Crucially, $\mathbb{E}[\tau_n \mid \mathcal{F}_n] = \hat{\tau}_n$, so the drift depends only on quantities available at decision time.

Step 3: Capacity region under quantization. For the quantized system with grid $\mathcal{G}_\delta$, the achievable service rates are

$$\mu_\delta^\star := \max_{L \in \mathcal{G}_\delta} \frac{kL}{L + f(k, L)} \;\ge\; \mu^\star - O(\delta),$$

since the optimal continuous threshold $L^\star$ satisfies $|L^\star - Q_\delta(L^\star)| < \delta$, and the service rate function $kL/\tau(L)$ is Lipschitz.

Step 4: Comparison with stabilizing policy. When $\Lambda < \mu_\delta^\star$, there exists $\varepsilon > 0$ and a stationary policy π_ε on the quantized grid such that $\mathbb{E}_{\pi_\varepsilon}[\mathbf{1}\{i \in S_n\}\, \ell_i(L_n)] \ge (\lambda_i^{\text{pkt}} + \varepsilon)\, \mathbb{E}_{\pi_\varepsilon}[\hat{\tau}_n]$. Since R-RDPP maximizes the score $\Phi_V^\eta(L)$ over $L \in \mathcal{G}_\delta$, it achieves a drift-plus-penalty ratio no worse than π_ε, yielding

$$\mathbb{E}[\Delta_n^{\text{R-RDPP}} \mid \mathcal{F}_n] \le \Gamma - \eta' \sum_i Q_i^+(T_n)$$

for constants $\Gamma < \infty$ and $\eta' > 0$.

Step 5: Foster–Lyapunov conclusion. The embedded chain $\{\boldsymbol{Q}^+(T_n)\}$ satisfies the same irreducibility, aperiodicity, and negative-drift conditions as in Lemma 4.1. The bounded output support ($\Delta_g < \infty$) and bounded input support ensure all frame-level quantities remain bounded. By the Foster–Lyapunov theorem, the chain is positive recurrent with finite stationary backlog. For the continuous input-length extension, the state space is uncountable, but the Meyn–Tweedie theory for general state spaces applies: bounded frame lengths ensure the existence of petite sets, and the i.i.d. arrival density provides ψ-irreducibility (see [21]). The same drift condition yields positive Harris recurrence. □

F Proof of Theorem 5.2

The proof decomposes the optimality gap into three components: an algorithmic gap $O(1/V)$, an information gap $O(\Delta_g^2)$, and a discretization gap $O(\delta)$. We establish each through a series of intermediate results.

Lemma F.1 (Expected Frame Cost with Output Variance). *Let $C_n := \sum_{t=T_n}^{T_{n+1}-1} \sum_i Q_i(t)$ denote the total cost during frame n. Then*

$$\mathbb{E}[C_n \mid \mathcal{F}_n] = \hat{\tau}_n S_0^{(n)} - (\hat{\tau}_n - 1)\, d_n + \frac{\Lambda\, \hat{\tau}_n(\hat{\tau}_n - 1)}{2} + \frac{\Lambda\, Var(\tau_n \mid \mathcal{F}_n)}{2},$$

where $S_0^{(n)} = \sum_i Q_i^+(T_n)$ and $d_n = \sum_{i \in S_n} \ell_i^{in}$.

Proof. By Lemma C.3, replacing the deterministic τ_n with random τ_n:

$$\mathbb{E}[C_n \mid \mathcal{F}_n] = \mathbb{E}[\tau_n \mid \mathcal{F}_n]\, S_0^{(n)} - (\mathbb{E}[\tau_n \mid \mathcal{F}_n] - 1)\, d_n + \frac{\Lambda}{2}\, \mathbb{E}[\tau_n(\tau_n - 1) \mid \mathcal{F}_n].$$

Using $\mathbb{E}[\tau_n^2 \mid \mathcal{F}_n] = \hat{\tau}_n^2 + \mathrm{Var}(\tau_n \mid \mathcal{F}_n)$:

$$\mathbb{E}[\tau_n(\tau_n - 1) \mid \mathcal{F}_n] = \hat{\tau}_n^2 + \mathrm{Var}(\tau_n \mid \mathcal{F}_n) - \hat{\tau}_n = \hat{\tau}_n(\hat{\tau}_n - 1) + \mathrm{Var}(\tau_n \mid \mathcal{F}_n).$$

Substituting yields the result. □

Proof of Theorem 5.2. **Step 1: Drift-plus-penalty bound with variance.** Combining Lemma C.2 (adapted for random τ_n) and Lemma F.1, for any $V > 0$:

$$\begin{aligned}
&\mathbb{E}[\Delta_n \mid \mathcal{F}_n] + V\,\mathbb{E}[C_n \mid \mathcal{F}_n] \\
&\le \tilde{B}_0' + \frac{V\Lambda\,\mathrm{Var}(\tau_n \mid \mathcal{F}_n)}{2} + V\Lambda S_0^{(n)} \\
&\quad + \hat{\tau}_n\left(\sum_i Q_i^+(T_n)\lambda_i^{\mathrm{pkt}} + \frac{V\Lambda(\hat{\tau}_n - 1)}{2}\right) \\
&\quad - \sum_{i\in S_n}\left(Q_i^+(T_n) + V(\hat{\tau}_n - 1)\right)\ell_i^{\mathrm{in}}.
\end{aligned} \tag{35}$$

Since $\mathrm{Var}(\tau_n \mid \mathcal{F}_n) \le \Delta_g^2$, the variance term contributes a per-frame constant $V\Lambda\Delta_g^2/2$.

Step 2: Oracle comparison. Let $\pi^\star$ be an oracle policy that observes output lengths at scheduling time and achieves optimal average cost $c^\star$. Fix $\varepsilon > 0$ and let $\pi_\varepsilon^\star$ achieve cost $c^\star + \varepsilon$. Under $\pi_\varepsilon^\star$, the bound (35) yields

$$\Delta_n^{\pi_\varepsilon^\star} + V\,\mathbb{E}[C_n^{\pi_\varepsilon^\star} \mid \mathcal{F}_n] \le \tilde{B}_0' + \frac{V\Lambda\Delta_g^2}{2} + V(c^\star + \varepsilon)\,\mathbb{E}[\tau_n^{\pi_\varepsilon^\star} \mid \mathcal{F}_n].$$

Step 3: R-RDPP achieves the same bound. R-RDPP maximizes $\Phi_V^\eta(L)$ over $L \in \mathcal{G}_\delta$, and the oracle operates on the continuous space. The score maximization ensures that R-RDPP's action-dependent terms are at least as good as those of $\pi_\varepsilon^\star$ restricted to $\mathcal{G}_\delta$. The discretization introduces an additional error of $O(\delta)$ per frame from two sources:

(i) Service amount error: The quantized length $Q_\delta(\ell^{\mathrm{in}})$ differs from the true ℓ^{in} by at most δ. Across k stations, the per-frame service error is at most $k\delta$.

(ii) Frame length error: The true frame length may differ from the quantized prediction by at most $(1 + L_f)\delta$, where L_f is the Lipschitz constant of f in its second argument.

Thus:

$$\Delta_n^{\text{R-RDPP}} + V\,\mathbb{E}[C_n^{\text{R-RDPP}} \mid \mathcal{F}_n] \le \tilde{B}_0' + \frac{V\Lambda\Delta_g^2}{2} + V(c^\star + \varepsilon)\,\mathbb{E}[\tau_n \mid \mathcal{F}_n] + V\,R_\delta, \tag{36}$$

where $R_\delta = O(\Lambda k(1 + L_f)\delta)$ captures the per-frame discretization cost.

Step 4: Telescoping over frames. Sum (36) over $n = 0, \ldots, N(T) - 1$ and take expectations. Using $\mathcal{L} \ge 0$:

$$\begin{aligned}
V\sum_{n=0}^{N(T)-1}\mathbb{E}[C_n] &\le \mathcal{L}(\boldsymbol{Q}^+(0)) + \left(\tilde{B}_0' + \frac{V\Lambda\Delta_g^2}{2} + VR_\delta\right)\mathbb{E}[N(T)] \\
&\quad + V(c^\star + \varepsilon)\,T.
\end{aligned}$$

Step 5: Normalization and limit. Divide by VT and take $T \to \infty$. By the renewal reward theorem, $\mathbb{E}[N(T)]/T \to 1/\mathbb{E}[\tau]$ where $\mathbb{E}[\tau] \ge \underline{\tau} := \min_{L,\ell^{\mathrm{out}}}[L + f(k, L) + g(k, L, \ell^{\mathrm{out}})] \ge 1$. We obtain:

$$\bar{c}^{\text{R-RDPP}} \le (c^\star + \varepsilon) + \frac{\tilde{B}_0'}{V\,\underline{\tau}} + \frac{\Lambda\Delta_g^2}{2\underline{\tau}} + \frac{R_\delta}{\underline{\tau}}.$$

Taking $\varepsilon \downarrow 0$ and defining $B_1 := \tilde{B}_0'/\underline{\tau}$, $B_2 := \Lambda\Delta_g^2/(2\underline{\tau})$, and $B_3(\delta) := R_\delta/\underline{\tau} = O(\delta)$:

$$\bar{c}^{\text{R-RDPP}} \le c^\star + \frac{B_1}{V} + B_2 + B_3(\delta), \tag{37}$$

which is equivalent to $c^\star + O(1/V) + O(\Delta_g^2) + O(\delta)$.

Explicit constants.

$$\begin{aligned}
B_1 &= \frac{1}{2\underline{\tau}}\sum_{i=1}^{N}\left[\bar{\tau}\,\mathbb{E}[(\ell_i^{\mathrm{in}})^2] + \bar{\tau}^2(\lambda_i^{\mathrm{pkt}})^2 + (L_I^{\mathrm{in}})^2\right], \\
B_2 &= \frac{\Lambda\,\Delta_g^2}{2\underline{\tau}}, \\
B_3(\delta) &= O\big(\Lambda\,k\,(1 + L_f)\,\delta\big),
\end{aligned}$$

where $\bar{\tau}$ and $\underline{\tau}$ are the maximum and minimum possible frame lengths.

Irreducibility of the information gap. The term $B_2 = O(\Delta_g^2)$ is irreducible by any causal policy. To see this, observe that the frame cost involves $\mathbb{E}[\tau_n^2 \mid \mathcal{F}_n] = \hat{\tau}_n^2 + \mathrm{Var}(\tau_n \mid \mathcal{F}_n)$, and the variance $\mathrm{Var}(\tau_n \mid \mathcal{F}_n) \le \Delta_g^2$ arises from the fundamental unpredictability of output lengths

at scheduling time. At the decision epoch, output lengths are unknown, so any causal policy that selects batch S_n based on $\mathcal{F}_n$ incurs a frame cost whose expectation includes the non-negative variance term $\frac{\Delta}{2}\mathrm{Var}(\tau_n \mid \mathcal{F}_n)$. Since this variance depends on the output-length distribution (not the policy), no causal scheduling rule can eliminate it. When $\Delta_g = 0$ (i.e., g is independent of ℓ^{out}, or the TTFT-only model with $g \equiv 0$), the information gap vanishes and we recover the standard RDPP bound of Theorem 4.2.

$O(1)$ **backlog scaling.** As a corollary, the time-average backlog satisfies

$$\limsup_{T\to\infty} \frac{1}{T}\sum_{t=0}^{T-1} \mathbb{E}\left[\sum_{i=1}^{N} Q_i(t)\right] = \bar{c}^{\text{R-RDPP}} = O(1),$$

confirming that stability and delay performance are preserved even in the presence of output uncertainty and continuous input lengths. □